

ZK-ACE: Identity-Centric Zero-Knowledge Authorization for Post-Quantum Blockchain Systems

Jian Sheng Wang
Yeah LLC

May 19, 2026

Abstract

Post-quantum signature schemes impose kilobyte-scale on-chain artifacts. Verifying them inside ZK circuits merely relocates the cost via expensive lattice arithmetic in prover circuits.

We present ZK-ACE (**Z**ero-**K**nowledge **A**uthorization for **C**ryptographic **E**ntities), which replaces transaction-carried signature objects with identity-bound ZK statements. Given a *deterministic identity derivation primitive* (DIDP) as a black box, the prover demonstrates in zero knowledge that an identity consistent with an on-chain commitment authorized the transaction—no signature object is produced or verified on-chain.

We provide game-based definitions and reduction-based proofs for authorization soundness, replay resistance, substitution resistance, and cross-domain separation, under knowledge soundness, collision resistance, and DIDP recovery hardness. Structural data accounting shows an order-of-magnitude reduction in per-transaction authorization data versus direct PQC deployment. A reference implementation offers two backends: Circle STARK (341 active rows / 361 AIR constraint expressions, 14.5 ms prove, 1.1 ms verify, ≈ 107 KB proofs, transparent setup, post-quantum-oriented) and Groth16/BN254 (2,155 R1CS constraints, 37.3 ms prove, 128-byte proofs). Both are roughly $500\text{--}2,300\times$ smaller than in-circuit PQC signature verification. Under mandatory per-block STARK aggregation, per-transaction consensus-visible data is ≈ 160 bytes.

Keywords. Post-Quantum Cryptography, Deterministic Identity, Zero-Knowledge Proofs, Identity Commitment, Blockchain Authorization, Circle STARK, Groth16, Account Abstraction.

Version note. This arXiv version consolidates the March–May 2026 revisions: unified replay-circuit benchmarks, dual-backend implementation results, squeeze2 sponge optimization, and batch STARK measurements.

1 Introduction

1.1 Post-Quantum Signatures and Blockchain Scalability

The standardization of post-quantum cryptography (PQC) by NIST represents a necessary step toward long-term security in the presence of quantum adversaries. Lattice-based signature schemes such as ML-DSA (formerly Dilithium) [3] and hash-based schemes such as SLH-DSA (formerly SPHINCS⁺) [4] have emerged as leading candidates to replace classical public-key signatures.

While these schemes are well-suited for conventional communication settings, their direct application to blockchain systems exposes a fundamental tension between post-quantum security

and scalability. A distinguishing characteristic of lattice-based signatures is their *large signature size*, typically on the order of several kilobytes. In blockchain environments, where transaction validity must be verified by all consensus participants and stored as part of the permanent ledger, such signature sizes translate directly into increased on-chain data growth. ML-DSA signatures at NIST security level 2 are approximately 2.4 KB, compared to 64 bytes for Ed25519 or 71 bytes for secp256k1 ECDSA—a roughly 30–40 $\times$ increase per transaction.

This pressure is further amplified in modern scaling architectures. In rollup-based systems, transaction data is posted as calldata or blob data to the base layer, where bandwidth and availability are explicitly constrained and economically priced [18]. Large post-quantum signatures therefore reduce effective transaction throughput and increase marginal costs per transaction. Crucially, this scalability pressure arises from the *structure of the authorization artifacts themselves*, rather than from inefficiencies in any particular implementation.

1.2 Limitations of ZK-Compressed Signature Verification

A natural response to the data-expansion problem is to move post-quantum signature verification off-chain and replace it with zero-knowledge proofs. Under this approach, a prover generates a zero-knowledge proof attesting to the validity of a post-quantum signature, and the blockchain verifies only the succinct proof [19].

However, this strategy encounters a critical limitation: verifying lattice-based signatures requires arithmetic over high-dimensional algebraic structures, which translates into a large number of constraints when expressed inside zero-knowledge circuits. Even when technically feasible, embedding lattice verification logic into ZK circuits significantly increases the proving cost and often shifts the scalability burden from on-chain data to off-chain computation and verification [10, 8].

As a result, ZK-verification of post-quantum signatures risks becoming a performance bottleneck in its own right. Rather than addressing the root cause of the scalability issue, this approach preserves the signature-centric model of authorization and merely relocates its computational cost. This observation motivates a more fundamental question: *should post-quantum signature verification be treated as a mandatory component of on-chain authorization at all?*

1.3 Key Observation: Authorization versus Signature Objects

At the consensus layer, blockchains do not inherently require verification of a specific cryptographic signature object. What consensus requires is assurance that a given transaction was *authorized* by the correct entity under the system’s rules.

In traditional designs, cryptographic signatures serve as the mechanism by which authorization is expressed and verified. However, signatures are ultimately an *implementation artifact* rather than the authorization semantics themselves. Treating the signature object as the primary unit of verification implicitly couples consensus security to the concrete properties of a particular signature scheme—including its key sizes, signature sizes, and verification costs.

This work adopts a different viewpoint: authorization should be modeled as a *semantic property*—that a specific identity has approved a specific transaction—independent of the concrete cryptographic artifact traditionally used to express that approval. From this perspective, the role of zero-knowledge proofs is not to verify the correctness of a post-quantum signature object, but to prove that the underlying authorization condition holds.

1.4 Our Setting and Scope

We consider a setting in which a *deterministic identity derivation primitive* (DIDP) is assumed to exist as a cryptographic building block.¹ A DIDP provides a stable, deterministic mapping from a 256-bit Root Entropy Value (REV) to context-specific cryptographic keys, potentially across multiple cryptographic schemes, including post-quantum constructions. Its security properties—including determinism, context isolation, and resistance to identity-root recovery—are taken as given. In this respect, a DIDP is treated analogously to other commonly assumed primitives such as verifiable random functions (VRFs), key derivation functions (KDFs), or cryptographic commitment schemes, which are routinely used as building blocks without re-proving their security in every application.

The present work does not propose, analyze, or modify any specific DIDP construction. Our contribution begins at the layer above the DIDP: given a deterministic identity derivation primitive, how should transaction authorization be represented, proven, and verified in a post-quantum-ready blockchain system?

Terminology: post-quantum-ready vs. post-quantum-secure. This paper uses two related but distinct terms. *Post-quantum-secure* refers to a concrete instantiation whose security holds against quantum adversaries under standard assumptions (e.g., hash preimage resistance under Grover-style quadratic speedup, and collision resistance against quantum algorithms; note that collision resistance is more aggressively degraded—roughly $2^{n/3}$ queries under the BHT algorithm [1]—so the adequate-output-length requirement is stronger for collision than for preimage). *Post-quantum-ready* refers to an architectural property: the system is designed so that a migration to post-quantum-secure components requires no structural changes to the protocol (e.g., no identity rotation, no circuit redesign). ZK-ACE is *post-quantum-ready* by design; its Circle STARK instantiation is post-quantum-oriented and should be regarded as post-quantum-secure only under adequate hash/output-length parameters and the proof-system assumptions stated in Section 3.4.

1.5 Contributions

This paper makes the following contributions:

1. We introduce ZK-ACE, a zero-knowledge authorization layer built on top of deterministic identity derivation, in which transaction authorization is proven via zero-knowledge proofs rather than explicit post-quantum signature objects.
2. We propose an on-chain data model consisting of a compact identity commitment and per-transaction zero-knowledge authorization proofs, replacing kilobyte-scale signature artifacts with constant-size verification material.
3. We formally specify the zero-knowledge circuit statement underlying ZK-ACE, including its public inputs, private witnesses, and five core constraints.
4. We define game-based security notions—authorization soundness, replay resistance, substitution resistance, and cross-domain separation—and provide reduction-based security arguments under standard assumptions (knowledge soundness, collision resistance, DIDP identity-root recovery hardness).
5. We describe two replay-protection mechanisms (nonce-registry and nullifier-set) with a formal analysis of their respective trade-offs.
6. We present batching and recursion strategies for high-throughput systems, while treating constant-verification recursive aggregation as future work beyond the current benchmarked

¹ACE-GF [2] is one concrete instantiation that satisfies the DIDP interface and security properties defined in Section 3.2. Any framework providing the same interface—a deterministic mapping from a high-entropy root to context-specific keys with the properties of determinism, context isolation, and identity-root recovery hardness—may be used in its place.

implementation.

7. We provide a structural, protocol-level accounting of on-chain authorization data requirements, demonstrating an order-of-magnitude reduction relative to signature-based post-quantum authorization.
8. We report a reference implementation with two pluggable backends—Circle STARK (Stwo, transparent and hash-based, with post-quantum security conditional on the instantiated parameters) and Groth16/BN254 (compact classical proofs)—with Criterion.rs benchmarks for the unified replay-circuit path. The STARK backend achieves 14.5 ms proving and 1.1 ms verification (341 active rows / 361 AIR constraint expressions, 11 permutations); the Groth16 backend achieves 37.3 ms proving and 1.6 ms verification over 2,155 R1CS constraints with 128-byte compressed proofs. Under deployment models with mandatory per-block STARK aggregation and on-chain aggregate verification, per-transaction consensus-visible data is ≈ 160 bytes (public inputs only) plus amortized aggregation proof overhead shared across all transactions in the block. Under a builder-off-path model, the per-transaction consensus-visible footprint is likewise ≈ 160 bytes, with aggregate proof verification handled by the block builder rather than the chain consensus. We provide a quantitative comparative analysis showing a roughly 500–2,300 $\times$ constraint reduction relative to in-circuit post-quantum signature verification.

1.6 Design Goals

The authorization model should satisfy the following properties:

- **Security.** Authorization must be cryptographically sound and resistant to classical and quantum adversaries within the assumed threat model.
- **Verifiability.** Transaction validity must be efficiently verifiable by consensus participants using only public information.
- **Scalability.** On-chain authorization data and verification cost should remain constant with respect to the size of post-quantum signature objects.
- **Migratability.** Existing identities should be able to transition to the new authorization model without disrupting established recovery or identity assumptions. Identity commitments should be proof-system-agnostic so that verifier and prover upgrades do not require identity rotation.
- **Aggregatability.** The authorization mechanism should admit batching today and remain compatible with recursive composition for high-throughput systems.

1.7 Paper Organization

Section 2 surveys related work. Section 3 introduces notation, the assumed DIDP interface, and the threat model. Section 4 describes the system model and transaction flow. Section 5 defines the identity commitment and binding rules. Section 6 clarifies the authorization semantics. Section 7 specifies the ZK circuit. Section 8 presents formal security definitions and proofs. Section 9 discusses on-chain verification and deployment. Section 10 provides the structural data accounting. Section 11 describes scalability extensions. Section 12 reports a reference implementation and benchmarks. Section 13 addresses limitations, and Section 14 concludes.

2 Related Work

Post-quantum signatures on blockchains. Several efforts have investigated the direct integration of lattice-based or hash-based signatures into blockchain transaction validation [3, 4, 5]. The Ethereum community has explored PQC migration paths within its long-term roadmap. The primary obstacle recognized in these efforts is the significant per-transaction data overhead.

Our work addresses this obstacle at the protocol architecture level rather than attempting to optimize signature parameters.

ZK-compressed signature verification. An alternative line of research embeds classical or post-quantum signature verification inside zero-knowledge circuits. Notable examples include Circom-based ECDSA verification [19] and explorations of lattice signature verification within SNARK circuits. While technically feasible for classical signatures whose verification is algebraically lightweight, extending this approach to lattice-based schemes significantly inflates the constraint count and prover cost. ZK-ACE avoids this entirely by not placing any signature verification logic inside the circuit.

Transparent and hash-based proof systems. STARKs [10] and FRI-based interactive oracle proofs provide transparent, plausibly post-quantum-secure proof generation without a trusted setup. Recent work by Chiesa, Fenzi, and Weissenberg [26] constructs zero-knowledge IOPs for constrained interleaved codes with near-optimal $(1 + o(1))$ soundness-to-proof-length overhead, further advancing the efficiency of hash-based proof systems. These developments improve the *proof system layer* and are complementary to ZK-ACE, which operates at the *authorization model layer*. Even with an optimal transparent proof system, embedding lattice-based signature verification into a ZK circuit still requires expressing high-dimensional polynomial arithmetic (NTTs, non-native modular reductions) as arithmetic constraints—yielding circuits on the order of millions of constraints. ZK-ACE sidesteps this bottleneck entirely: its circuit requires only a small number of ZK-friendly hash evaluations (2,155 R1CS constraints in the Groth16 backend, 341 active rows / 361 AIR constraint expressions in the Circle STARK backend; see Section 7.3), and is compatible with any backend proof system, including hash-based and transparent constructions. The reference implementation includes a Circle STARK backend built on Stwo [27], demonstrating that the ZK-ACE circuit can be instantiated with a transparent hash-based proof system with competitive performance, subject to the concrete security margins discussed in Section 13.

Commitment and nullifier systems. Privacy-focused protocols such as Zcash Sapling [14] and Tornado Cash [15] employ commitment-based state models with nullifier-set replay protection. Semaphore [16] uses identity commitments for anonymous signaling. ZK-ACE draws on the commitment/nullifier design vocabulary but applies it to a distinct problem: *authorization* of transactions against a committed deterministic identity, rather than private value transfer or anonymous membership proofs.

Account abstraction and modular validation. ERC-4337 [17] introduces account abstraction for Ethereum, allowing accounts to define custom validation logic via validator modules. ZK-ACE can be instantiated as such a module, replacing signature-object checks with zero-knowledge proof verification. Unlike generic account-abstraction validators, ZK-ACE provides a complete authorization framework with formal security guarantees.

Identity-based cryptography. Identity-based encryption [20] and identity-based signatures [21] bind cryptographic capabilities to identity strings, typically requiring a trusted key-generation center. ZK-ACE does not assume any trusted third party; identity is self-sovereign and deterministically derived from user-controlled root material via a DIDP.

3 Preliminaries and Assumptions

3.1 Notation

We use the following notation throughout.

- λ : security parameter.
- $H(\cdot)$: a cryptographic hash function. When used inside zero-knowledge circuits, H is instantiated with a ZK-friendly hash (e.g., Poseidon [7]); outside circuits it may be a conventional hash.
- $\parallel$: concatenation with unambiguous parsing (e.g., length-prefixed encoding).
- **REV**: 256-bit Root Entropy Value serving as the identity root, derived via the DIDP. Ephemeral; never persistently stored.
- **Ctx**: derivation context tuple ($\text{AlgID}, \text{Domain}, \text{Index}$) used by the DIDP to derive context-specific keys from REV.
- ID_{com} : on-chain identity commitment.
- **TxHash**: hash of all authorization-relevant transaction fields.
- **domain**: chain or application domain separator.
- **target**: target-binding digest (address, call-data hash, or account context).
- **rp_com**: replay-prevention commitment (nonce commitment or nullifier).
- **PPT**: probabilistic polynomial-time.
- $\text{negl}(\lambda)$: a function negligible in λ .
- $x \xleftarrow{\$} S$: x sampled uniformly at random from set S .

3.2 Assumed DIDP Interface (Black-Box)

We assume the existence of a *deterministic identity derivation primitive* (DIDP), treated as a black-box building block. Rather than fixing a specific construction, we formalize only the interface and properties required by the zero-knowledge authorization layer. Any concrete framework satisfying this interface may be used; ACE-GF [2] is one such instantiation.

At the core of the DIDP is a high-entropy identity root called the *Root Entropy Value* (REV), a 256-bit value that uniquely defines the cryptographic identity of an entity and serves as the sole source of entropy for all derived keys. The REV exists only ephemerally in memory during execution and is never persistently stored or exported.

The DIDP provides two principal operations:

- **Identity reconstruction**: $\text{REV} \leftarrow \text{Unseal}(\text{params}, \text{SA}, \text{Cred})$, where **SA** is a sealed artifact (an encrypted encoding of REV) and **Cred** is an authorization credential. Returns REV if authentication succeeds, or $\perp$ otherwise. The credential controls *access* to the identity root but does not influence key derivation.
- **Context-specific key derivation**: $\text{Key} \leftarrow \text{Derive}(\text{REV}, \text{Ctx})$, where $\text{Ctx} = (\text{AlgID}, \text{Domain}, \text{Index})$ is a context descriptor encoding the target cryptographic algorithm (e.g., Ed25519, Secp256k1, ML-DSA), the application domain (e.g., signing, encryption), and a derivation index. The context tuple enforces cryptographic isolation: keys derived under distinct context tuples are computationally independent.

This two-operation interface cleanly separates the *authorization pipeline* (controlling access to REV via sealed artifacts and credentials) from the *identity pipeline* (deterministically deriving keys from REV using context-isolated derivation). The zero-knowledge authorization layer interacts only with the identity pipeline: it assumes the prover has obtained REV (via Unseal) and uses REV together with a context tuple Ctx to derive the relevant cryptographic material.

3.3 Security Properties Assumed from the DIDP

The zero-knowledge authorization layer relies on the following security properties of the assumed DIDP interface:

- **Determinism:** For fixed inputs, Derive produces identical outputs. In particular, the same (REV, Ctx) pair always yields the same derived key.
- **Context isolation:** Distinct context tuples $\text{Ctx} \neq \text{Ctx}'$ yield computationally independent derived keys. This ensures that compromise of a key derived under one context does not enable recovery of keys derived under other contexts, nor does it reveal information about the underlying REV.
- **Identity-root recovery hardness:** We formalize this property via the following game $\text{Game}_{\mathcal{A}}^{\text{rec-didp}}(\lambda)$:
 1. The challenger samples $\text{REV} \xleftarrow{\$} \{0, 1\}^{256}$.
 2. The adversary $\mathcal{A}$ is given oracle access to $\mathcal{O}_{\text{pub}}$, which on input $(\text{salt}, \text{domain})$ returns $H(\text{REV} \parallel \text{salt} \parallel \text{domain})$, and to $\mathcal{O}_{\text{derive}}$, which on input Ctx returns $H(\text{Derive}(\text{REV}, \text{Ctx}))$. These oracles model the ability to observe identity commitments and derived public identifiers without learning REV itself.
 3. $\mathcal{A}$ outputs a candidate REV^* .
 4. $\mathcal{A}$ wins if $\text{REV}^* = \text{REV}$.

For any PPT adversary $\mathcal{A}$:

$$\text{Adv}_{\mathcal{A}}^{\text{rec-didp}}(\lambda) = \Pr[\text{REV}^* = \text{REV}] \leq \text{negl}(\lambda).$$

Informally, no efficient adversary can recover the identity root from public outputs alone.

We note that identity-root recovery hardness is the *minimal* assumption required by the authorization soundness proof (Theorem 8.1). The proof’s case analysis (Section 8.1) shows that any successful forgery must either produce a hash collision (handled by collision resistance of H) or yield the correct REV from the extracted witness (handled by recovery hardness). A stronger notion—such as infeasibility of forging $H(\text{Derive}(\text{REV}, \text{Ctx}))$ for a fresh context Ctx without knowing REV—would follow from recovery hardness composed with the collision resistance of H , but is not needed as a separate assumption.

- **Multi-algorithm isolation:** Keys derived for different cryptographic algorithms (via distinct AlgID values in Ctx) from the same REV are computationally isolated, such that compromise of one derived key does not enable derivation of others.

No additional assumptions are made regarding the internal structure of the DIDP. In particular, this work does not depend on any specific key derivation function (e.g., HKDF [22]), memory-hard password hashing scheme (e.g., Argon2 [6]), or mnemonic encoding used within the instantiation.

3.4 Zero-Knowledge Proof System Model

The constructions in this paper assume a general-purpose zero-knowledge proof system supporting succinct non-interactive arguments of knowledge, building on the foundational framework of interactive proofs [23]. The specific proving system is not fixed and may be instantiated using SNARK-based [9], PLONK-based [8], STARK-based [10], or IPA-based constructions [11]. We assume the following standard properties:

- **Completeness:** An honest prover with a valid witness can always produce an accepting proof.
- **Knowledge soundness:** For any PPT prover $\mathcal{P}^*$ that produces an accepting proof with non-negligible probability, there exists a PPT extractor $\mathcal{E}$ that, given access to $\mathcal{P}^*$, outputs a valid witness with overwhelming probability. Formally,

$$\Pr[\text{Verify}(\text{vk}, \pi, x) = 1 \wedge \mathcal{E}^{\mathcal{P}^*}(x) \text{ fails}] \leq \text{negl}(\lambda).$$

- **Zero-knowledge:** Proofs reveal no information about the private witness beyond the validity of the statement.
- **Public-input binding:** The proof is cryptographically bound to the declared public inputs. For any PPT adversary,

$$\Pr[\text{Verify}(\text{vk}, \pi, x) = 1 \wedge \text{Verify}(\text{vk}, \pi, x') = 1 \wedge x \neq x'] \leq \text{negl}(\lambda).$$

- **Succinctness:** Proof size and verification complexity are independent of the size of the witness.

The precise extraction model depends on the chosen proof system: pairing-based SNARKs (e.g., Groth16) typically achieve knowledge soundness in the algebraic group model or via non-black-box extraction, while IOP-based systems (e.g., STARKs, PLONK) rely on point-query extraction from the underlying interactive oracle proof. Our security reductions are stated generically in terms of the knowledge-soundness advantage Adv^{ks} and are compatible with any proof system satisfying the above properties under its respective extraction model.

Post-quantum security stratification. The two reference backends differ fundamentally in their quantum security posture, and this distinction applies to all security statements in this paper:

- **Circle STARK backend (Stwo, post-quantum).** Knowledge soundness reduces to FRI soundness and collision resistance of the underlying hash function (Poseidon2 over Mersenne-31). Post-quantum security of these assumptions is conditional on parameter choices that provide adequate quantum margins for both preimage resistance (Grover-style $2^{n/2}$ speedup) and collision resistance (BHT-style $2^{n/3}$ speedup); the collision margin is more demanding and the concrete M31 Poseidon2 parameters require dedicated quantum cryptanalysis to confirm the desired security level. Under the assumption that the instantiated hash function is collision-resistant against quantum adversaries, the security theorem statements of Section 8 admit a quantum-adversary interpretation, provided the underlying proof system achieves knowledge soundness against QPT adversaries and the random oracle is programmable in the quantum setting (QROM); establishing these conditions formally is beyond the scope of this work.
- **Groth16/BN254 backend (classical only).** Knowledge soundness relies on the algebraic group model and the discrete logarithm hardness on the BN254 elliptic curve. Both assumptions are broken by Shor’s algorithm on a sufficiently capable quantum computer. **The Groth16 backend is therefore classical-only and must not be deployed in post-quantum threat models.** It is provided for EVM compatibility and circuit feasibility demonstration only.

Throughout this paper, the phrase “post-quantum-ready” refers to the architectural property that identity commitments and the circuit statement are proof-system agnostic, enabling migration from the Groth16 backend to a transparent hash-based backend without identity rotation. Claims about post-quantum security of a concrete Circle STARK deployment are conditional on the instantiated hash function, output length, FRI parameters, and quantum-adversary model.

3.5 Threat Model

We consider an adversary $\mathcal{A}$ with the following capabilities:

- Full visibility into on-chain state, transaction history, and mempool-visible submissions.
- The ability to submit, reorder, replay, and substitute transaction payloads and attempt proof reuse across contexts, within the limits of the consensus protocol.
- The ability to perform offline cryptographic attacks, including attacks enabled by future quantum computation, against traditional public-key primitives.

The adversary may attempt replay attacks by reusing authorization artifacts, substitution attacks by binding valid authorization material to unintended transactions, or linkage attacks by correlating authorization data across transactions or contexts.

We do *not* assume the presence of trusted hardware, trusted execution environments, or private mempools. All authorization artifacts are assumed to be publicly observable. Compromise of the underlying DIDP root material (REV) for an identity is out of scope and treated as key compromise of the base primitive.

Authorization intent and the DIDP gating role. A natural question is where *user intent* enters the model when no explicit signature object is produced. The answer lies in the DIDP Unseal operation: reconstructing REV requires the prover to supply a valid credential **Cred** (e.g., a password, biometric token, or hardware attestation accepted by the sealed artifact). Proof generation is therefore structurally gated on successful unsealing—the wallet cannot produce a valid authorization proof without first obtaining an ephemeral REV, which requires the user to present their credential. This is the mechanism by which authorization intent is anchored in ZK-ACE: the act of unsealing expresses the user’s willingness to authorize, and the zero-knowledge proof is the cryptographic evidence of that intent. Local wallet policy governing *which* transactions may trigger an unseal attempt (e.g., spending limits, confirmation dialogs, hardware-button confirmation) is outside the scope of this work and is treated as an implementation concern orthogonal to the protocol-level security guarantees.

4 System Model

4.1 Roles

ZK-ACE involves the following roles:

- **Prover.** The prover is the entity controlling a deterministic identity derived via the DIDP, typically a user or wallet application. The prover generates zero-knowledge proofs attesting that a given transaction has been authorized by the corresponding identity.
- **Verifier.** The verifier is the consensus-facing component of the blockchain system, such as an on-chain smart contract or a native verification rule. The verifier checks the validity of submitted proofs and enforces replay protection according to protocol rules.
- **Optional aggregator or relayer.** In scalable deployments, an optional aggregator or relayer may collect authorization requests from multiple provers, aggregate proofs, or submit batched authorization data to the chain. This role is not trusted and does not learn private identity material.

The security of ZK-ACE does not depend on the honesty of aggregators or relayers; they serve only as performance and usability optimizations.

4.2 On-Chain State

ZK-ACE minimizes on-chain state by storing only data strictly required for authorization verification and replay prevention:

- **Identity commitment.** For each identity participating in ZK-ACE, the chain stores a compact commitment ID_{com} to the underlying deterministic identity. This commitment serves as the primary on-chain identity anchor for all future authorization proofs associated with that identity.
- **Replay protection state.** Depending on the chosen replay-prevention model (Section 7.5), the chain additionally maintains:
 - a *nonce registry*, enforcing monotonic or uniqueness constraints on authorization nonces;
 - or
 - a *nullifier set*, marking authorization tokens as spent; or

Algorithm 1 ZK-ACE Transaction Authorization

Prover side:

- 1: Construct transaction payload tx and compute $\text{TxHash} \leftarrow H(\text{tx})$.
- 2: Using identity material derived via the DIDP, generate witness $w = (\text{REV}, \text{salt}, \text{Ctx}, \text{nonce}, \text{aux})$.
- 3: Compute public inputs $\pi_{\text{pub}} = (\text{ID}_{\text{com}}, \text{TxHash}, \text{domain}, \text{target}, \text{rp_com})$.
- 4: Generate zero-knowledge proof $\pi \leftarrow \text{Prove}(\text{pk}, w, \pi_{\text{pub}})$.
- 5: Submit $(\text{tx}, \pi, \pi_{\text{pub}})$ to the chain (directly or via a relayer).

Verifier side (given $\text{tx}, \pi, \pi_{\text{pub}}$):

- 6: **Context binding check:** Recompute $\text{TxHash}' \leftarrow H(\text{tx})$ from the submitted transaction payload and verify $\text{TxHash}' = \pi_{\text{pub}}.\text{TxHash}$. Reject if mismatch.
 - 7: **Public-input validation:** Verify that $\pi_{\text{pub}}.\text{ID}_{\text{com}}$ is a registered on-chain identity commitment, and that $\pi_{\text{pub}}.\text{domain}$ and $\pi_{\text{pub}}.\text{target}$ match the expected values for the transaction context. Reject if any mismatch.
 - 8: **Proof verification:** Verify $\text{ZKVerify}(\text{vk}, \pi, \pi_{\text{pub}}) \stackrel{?}{=} 1$. Reject if verification fails.
 - 9: **Replay check:** Verify replay predicate (nonce freshness or nullifier novelty). Reject if violated.
 - 10: If all checks pass, accept transaction and update replay state.
-

– an *authorization index* associated with the identity commitment.

- **Verifier parameters.** The verification key vk for the proof system.

No private keys, post-quantum signatures, or identity root material (REV) are stored on-chain.

4.3 Transaction Authorization Flow

The ZK-ACE authorization process for a single transaction proceeds as specified in Algorithm 1.

Throughout this process, no post-quantum signature objects are revealed or verified on-chain. Authorization validity is established solely through the correctness of the zero-knowledge proof and its binding to the committed identity and transaction hash.

End-to-end derivation invariant. For the system model to function correctly end-to-end, the circuit-native derivation function used inside the zero-knowledge proof must be *identical* to the external DIDP derivation function: same hash primitive, same encoding convention, same field-element representation. When this invariant holds, a proof attesting to circuit-native derivation correctness simultaneously attests to DIDP-level derivation correctness, and the authorization guarantee is end-to-end. The pipeline is: the prover unseals REV from the sealed artifact via Unseal , uses (REV, Ctx) to derive authorization material via the circuit-native Derive , binds all authorization-relevant fields into the proof witnesses, and submits a zero-knowledge proof. The verifier checks only public inputs and proof validity and never observes REV or any derived key. The formal statement of this requirement—and the consequence of violating it—is given in Section 7 as the canonical derivation requirement within constraint (C2).

5 Identity Commitment and Binding Model

5.1 Commitment Construction

For each deterministic identity participating in ZK-ACE, the chain stores a compact identity commitment defined as:

$$\text{ID}_{\text{com}} = H(\text{REV} \parallel \text{salt} \parallel \text{domain}), \quad (1)$$

where:

- **REV** is the 256-bit Root Entropy Value serving as the identity root, reconstructed ephemerally via the DIDP Unseal operation;
- $\text{salt} \xleftarrow{\$} \{0,1\}^\lambda$ is an identity-specific random or pseudorandom value;
- **domain** encodes domain-separation parameters, such as the target blockchain or application namespace.

The commitment serves as the primary on-chain identity anchor. Authorization additionally requires per-transaction public inputs (**TxHash**, **domain**, **target**, **rp_com**) and replay-prevention state, but ID_{com} is the persistent identity-binding component. Neither **REV** nor any derived keys are revealed on-chain.

The inclusion of **salt** serves multiple purposes. First, it enables *commitment re-randomization*, allowing the same underlying identity to be associated with multiple unlinkable commitments over time. Second, it provides a mechanism for *cross-chain or cross-domain isolation*, ensuring that identity commitments used in different execution environments are cryptographically distinct. Finally, salt-based randomization supports basic privacy controls by preventing trivial linkage between commitments derived from the same identity.

5.2 Public Input Binding Rules

Authorization proofs in ZK-ACE are verified against a set of public inputs that bind the proof to a specific transaction and execution context. These public inputs are consensus-visible and must be explicitly declared to the verifier. The following values are required:

1. **Transaction hash** (**TxHash**). Binds the authorization proof to a specific transaction payload, preventing substitution attacks.
2. **Chain or domain identifier** (**domain**). Ensures authorization proofs are valid only within the intended execution environment, preventing cross-chain replay.
3. **Derivation-target hash** (**target**). This is a hash commitment to context-derived authorization key material, i.e., $\text{target} = H(\text{Derive}(\text{REV}, \text{Ctx}))$, ensuring the proof is bound to a specific identity derivation context.
4. **Replay-prevention commitment** (**rp_com**). A nonce value, nullifier, or authorization-index commitment included to enforce single-use or monotonic authorization semantics.

Collectively, these public inputs ensure that each authorization proof is bound to *who* authorized the transaction (via ID_{com}), *which transaction* was authorized (via **TxHash**), *which derivation context* was used (via **target**), and *where and when* the authorization is valid (via **domain** and **rp_com**).

5.3 Linkability Considerations

ZK-ACE allows multiple identity commitments to be associated with the same underlying deterministic identity. This flexibility enables different trade-offs between privacy, usability, and auditability.

One approach is to maintain a single long-lived commitment per identity, maximizing simplicity and minimizing on-chain state. Alternatively, a prover may periodically rotate the salt value and register a new commitment, reducing linkability between authorization events at the cost of additional on-chain updates.

Importantly, linkability is controlled entirely at the commitment layer. Authorization proofs themselves reveal no additional information about the underlying identity beyond what is implied by the chosen commitment policy.

6 Authorization Semantics

6.1 What We Prove: Authorization

At a high level, ZK-ACE proves that a transaction is authorized by a committed identity under a deterministic derivation framework. The authorization statement proven in zero knowledge can be expressed as:

There exists a Root Entropy Value REV and associated derivation context such that:
(i) *the identity commitment derived from REV matches the on-chain commitment* ID_{com} , *and (ii) the identity anchored by REV has authorized the transaction with* *hash TxHash under the prescribed derivation context and replay-prevention rules.*

This statement is existential in nature. The zero-knowledge proof demonstrates the existence of a valid authorizing identity consistent with the on-chain commitment, without revealing REV or any derived keys. Authorization, in this model, is a *semantic property*: a transaction is valid if it can be shown that an identity anchored on-chain has approved the transaction according to the system’s deterministic authorization rules.

6.2 What We Do Not Prove

To avoid ambiguity, we explicitly state what ZK-ACE does *not* attempt to prove:

- **No verification of post-quantum signature objects.** The zero-knowledge circuit does not verify the correctness of any ML-DSA or other post-quantum signature. Signature objects are not provided as inputs to the circuit and do not appear on-chain.
- **No lattice arithmetic inside the circuit.** The circuit does not implement lattice-based cryptographic operations, nor does it encode the verification logic of any post-quantum signature scheme.
- **No compression of post-quantum signatures.** ZK-ACE does not claim to compress post-quantum signatures or reduce their size. Instead, it eliminates the need to place post-quantum signature material on-chain altogether.

The construction should therefore not be interpreted as a zero-knowledge optimization of post-quantum signatures. It represents a *shift in authorization semantics*, in which explicit signature verification is replaced by zero-knowledge proofs of deterministic authorization.

6.3 Authorization Token Definition

For concreteness, we define a unified authorization token that captures the semantic intent of transaction approval. Let the authorization token be:

$$\text{Auth} = H(\text{REV} \parallel \text{Ctx} \parallel \text{TxHash} \parallel \text{domain} \parallel \text{nonce}), \quad (2)$$

where each component binds the authorization to a specific identity root (REV), derivation context (Ctx), transaction (TxHash), execution domain (domain), and replay-prevention state (nonce).

The authorization token Auth is not a cryptographic signature and is not intended to be publicly verified outside the zero-knowledge proof. It serves as a conceptual representation of the authorization semantics enforced by the circuit. The zero-knowledge proof attests that such a token can be correctly derived from a committed identity root and that it binds the identity to the declared public inputs.

7 ZK Circuit Specification

This section specifies the zero-knowledge statement proven by ZK-ACE and the corresponding circuit structure. The circuit is designed to be compatible with standard constraint systems (e.g., R1CS or PLONK-like arithmetization) and to avoid embedding post-quantum signature verification logic.

7.1 Circuit Statement

Let ID_{com} be an on-chain identity commitment, $TxHash$ the transaction hash, $domain$ a chain/application domain tag, $target$ a derivation-target hash, and rp_com a replay-prevention commitment. The ZK-ACE circuit proves the following statement:

Prove knowledge of $(REV, salt, Ctx, nonce, aux)$ such that:

1. *the identity commitment recomputed from $(REV, salt, domain)$ equals the public commitment ID_{com} ;*
2. *the target binding is consistent with deterministic key derivation under (REV, Ctx) ;*
3. *the transaction hash $TxHash$ is authorized under the derived context and replay-prevention rule.*

This statement is existential: the proof establishes that *some* identity root consistent with ID_{com} authorized the specific transaction context, without revealing REV or any derived keys.

7.2 Witness and Public Inputs

Private witness (secret inputs). The witness consists of:

- **REV:** the 256-bit Root Entropy Value (identity root, reconstructed ephemerally via DIDP Unseal);
- **salt:** commitment salt associated with the identity;
- **Ctx:** derivation context tuple ($AlgID, Domain, Index$) specifying the target cryptographic algorithm, application domain, and derivation index;
- **nonce:** replay-prevention value (or material used to derive a nullifier);
- **aux:** auxiliary secret material that may be required by a specific circuit instantiation (e.g., intermediate absorption states for multi-round Poseidon calls, or padding values for variable-length input encoding). The core constraints (C1)–(C5) are stated without reference to aux ; it is included in the witness definition as an instantiation-reserved slot so that concrete deployments can carry implementation-specific intermediates without modifying the abstract witness structure.

Public inputs. The public inputs consist of:

- **ID_{com} :** on-chain identity commitment;
- **$TxHash$:** transaction hash to be authorized;
- **domain:** chain/application domain tag;
- **target:** hash commitment to the context-derived authorization key material, i.e., $target = H(Derive(REV, Ctx))$;
- **rp_com :** replay-prevention commitment (nonce commitment or nullifier).

All public inputs are consensus-visible and must be bound to the proof to prevent substitution and replay attacks.

7.3 Core Constraints

We describe the circuit constraints at a high level, in a form intended to map naturally to standard arithmetization. Let $H(\cdot)$ denote the selected in-circuit hash used for binding.

- (C1) **Commitment consistency.** The circuit recomputes the identity commitment and enforces equality with ID_{com} :

$$H(REV\|salt\|domain) = ID_{com}. \quad (3)$$

This ensures that the prover controls an identity root consistent with the on-chain anchor.

- (C2) **Deterministic derivation correctness (target binding).** The circuit enforces that the public target hash corresponds to the context-specific key derived from the identity root:

$$target = H(Derive(REV, Ctx)). \quad (4)$$

The outer hash ensures that the derived key itself is never exposed as a public input; only a hash commitment to it (**target**) is revealed on-chain, preserving the secrecy of identity-derived cryptographic material. The domain component of Ctx is constrained to equal the public input **domain** by constraint (C5), ensuring that derivation correctness is bound to the declared domain.

Implementation note: To minimize constraint count, **Derive** is instantiated inside the circuit as a deterministic sequence of ZK-friendly hashes (e.g., Poseidon), avoiding the overhead of simulating non-native field arithmetic or expensive conventional encodings (e.g., Bech32, Base58) inside the circuit. The context tuple $Ctx = (AlgID, Domain, Index)$ is encoded as a fixed-length field-element input to the hash.

Instantiation scope and canonical derivation requirement. The function **Derive** referenced in (C2) denotes the *circuit-native derivation function*—i.e., the concrete ZK-friendly hash sequence evaluated inside the proof system—rather than the external DIDP **Derive** implementation, which may use non-circuit-friendly primitives (e.g., HKDF, HMAC-SHA-256). The security analysis (Section 8) operates entirely on the circuit-native function: what the proof system enforces is that the prover knows a **REV** from which the declared **target** can be derived via this circuit-native function.

Canonical derivation requirement (deployment invariant). For the end-to-end security claim to hold—i.e., for a valid proof to imply that the *DIDP-registered* identity authorized the transaction—deployments *must* adopt the circuit-native derivation as the canonical implementation of **Derive** in the external DIDP. Concretely, the off-chain DIDP pipeline must use the same ZK-friendly hash function, encoding convention, and field-element representation as the circuit, ensuring that circuit-native $Derive(REV, Ctx)$ and external $Derive(REV, Ctx)$ produce identical outputs on all inputs. When this invariant holds, a proof attesting to circuit-native derivation correctness simultaneously attests to DIDP-level derivation correctness, and the reduction in Theorem 8.1 applies end-to-end. Violating this invariant—e.g., by using HKDF externally while the circuit uses Poseidon—would break the functional equivalence and invalidate the end-to-end security argument, even if each component is individually secure. ACE-GF [2] satisfies this invariant by construction, as its off-chain derivation is defined using the same Poseidon/Poseidon2 primitives as the reference circuit.

- (C3) **Authorization binding to TxHash.** The circuit binds the authorization to the specific transaction hash and context by enforcing the authorization token relation:

$$Auth = H(REV\|Ctx\|TxHash\|domain\|nonce). \quad (5)$$

- (C4) **Anti-replay (nonce/nullifier rule).** The circuit commits to replay-prevention state in a way that the verifier can enforce uniqueness or monotonicity. Two canonical forms are supported:

- *Nonce commitment:*

$$\text{rp_com} = H(\text{ID}_{\text{com}} \parallel \text{nonce}), \quad (6)$$

enabling per-identity nonce tracking. *Note on verifiability:* because `nonce` is a private witness, the chain cannot directly read the nonce value from `rp_com`. Enforcing strict monotonicity (e.g., expected nonce = $n + 1$) therefore requires either (a) making `nonce` a public input so the chain can compare it against its stored nonce state, or (b) treating `rp_com` as an opaque commitment and enforcing uniqueness only (i.e., rejecting any `rp_com` that has appeared before), which reduces to nullifier-set semantics. Deployments requiring strict monotonic nonce ordering should use option (a).

- *Nullifier (single-use token):*

$$\text{rp_com} = H(\text{Auth} \parallel \text{domain}), \quad (7)$$

enabling the chain to reject reuse by maintaining a nullifier set.

(C5) Domain separation and context consistency. The circuit enforces that the `domain` value declared as a public input is used consistently in all commitment and binding computations, preventing cross-chain or cross-application proof reuse. This includes two requirements:

- (a) *Public-input consistency:* `domain` appears as an explicit input to the commitment binding (C1), the authorization binding (C3), and the replay-prevention computation (C4).
- (b) *Context-domain binding:* The circuit enforces that the domain component of the context tuple matches the publicly declared domain:

$$\text{Ctx.Domain} = \text{domain}. \quad (8)$$

This constraint ensures that the derivation in (C2) operates under the same domain as all other bindings, closing the link between the context-specific derivation and the public domain identifier.

These constraints ensure that a valid proof simultaneously attests to identity ownership (via preimage knowledge of REV for `IDcom`), context correctness (via deterministic derivation under `Ctx`), transaction-specific authorization (via binding to `TxHash`), and replay prevention (via `rp_com`).

Constraint count estimation. The circuit cost is dominated by the in-circuit hash evaluations. The reference implementation provides two backends with different Poseidon parameterizations: (i) a Groth16/BN254 backend using a manual Poseidon implementation with width $t = 3$, S-box exponent $\alpha = 5$, 8 full rounds (4+4), and 56 partial rounds over the BN254 scalar field; and (ii) a Circle STARK backend (Stwo [27]) using Poseidon2 with width $t = 16$ and $\alpha = 5$ over the Mersenne-31 field.

Table 1 provides a logical five-relation baseline for the Groth16/BN254 backend.

The nonce-registry and nullifier-set semantics share the same post-derivation replay circuit and therefore produce identical constraint counts. Domain separation (C5) is enforced as an equality constraint on the domain component of `Ctx` and adds negligible overhead.

The Circle STARK backend compiles the same five-constraint circuit to 341 active rows (11 Poseidon2 permutations, `log_size=9`), 100 main trace columns, 34 preprocessed columns, and 361 AIR constraint expressions. The AIR representation encodes more logic per row than an R1CS multiplication gate, so the two constraint counts are not directly comparable, but both are extraordinarily compact by ZK circuit standards.

Table 1: Logical five-relation baseline for the ZK-ACE circuit (Groth16/BN254 backend, Poseidon $t = 3$, $\alpha = 5$).

Constraint	Inputs	Hash calls	R1CS
(C1) Commitment consistency	3	1	≈ 200
(C2) Derivation correctness	$4 + 1$	2	≈ 400
(C3) Authorization binding	7	1	≈ 400
(C4) Replay prevention	2	1	≈ 200
(C5) Domain sep. + <code>enforce_equal</code>	—	—	4
Total		5	$\approx 1,200$

For comparison, verifying a lattice-based signature such as ML-DSA inside a zero-knowledge circuit requires expressing: (i) Number Theoretic Transforms (NTTs) over degree-256 polynomial rings in $\mathbb{Z}_q$ with $q = 2^{23} - 2^{13} + 1$, each requiring $O(n \log n)$ multiplications in a non-native field; (ii) high-bit-width modular reductions (23-bit modulus emulated over a ≈ 254 -bit proof-system field, requiring range-check gadgets per limb); and (iii) rejection-sampling and hint-reconstruction logic with data-dependent branching. ML-DSA verification at NIST level 2 involves $k \times \ell = 4 \times 4$ matrix-vector NTTs and $k = 4$ polynomial reconstructions, yielding a structural lower bound on the order of millions of R1CS constraints even with optimized non-native arithmetic (see Table 10 in Section 12.3 for a detailed breakdown). The measured ZK-ACE circuit (2,155 R1CS / 341 AIR active rows; Table 2) is therefore **roughly 500–2,300 $\times$ smaller** than an in-circuit post-quantum signature verification. This gap reflects the structural difference between a small number of ZK-friendly hash invocations and the full algebraic machinery of lattice-based signature verification. The resulting circuit size makes proving feasible on consumer-grade hardware (≈ 14.5 ms with the STARK backend, ≈ 37.3 ms with Groth16; see Section 12) and enables practical batching while keeping recursive composition as a natural extension (Section 11).

7.4 Circuit Optimization via Sponge Multi-Output Squeeze

The five-constraint, five-hash-call baseline above follows a convention inherited from HMAC-style constructions, in which each logical binding step is protected by an independent hash invocation to defend against length-extension attacks. Length-extension attacks are a structural property of Merkle-Damgård hash functions (e.g., MD5, SHA-1, SHA-256). Poseidon and Poseidon2 use a sponge construction and are structurally immune to length-extension attacks; the outer wrapping implied by HMAC-style double-hashing is therefore unnecessary.

This observation, which emerged from discussion with Alberto La Rocca [33], who independently confirmed a 2-call lower bound for the core MAC computation in his implementation, motivates the implemented sponge optimization. The logical hash relation count is reduced from 5 to 3 while all security properties formalized in Section 8 are preserved.

C2: inner + outer $\rightarrow$ single multi-output squeeze. Constraint (C2) currently evaluates $\text{target} = H(H(\text{REV}, \text{Ctx}))$: an inner derivation hash followed by an outer commitment wrapper that prevents the derived key from being directly exposed as a public input. A Poseidon sponge supports multi-output *squeeze*: after absorbing (REV, Ctx) , two independent output lanes are produced in a single permutation pass. One lane serves as internal derived key material; the other is released as the public **target**. The capacity portion of the sponge state ensures that the internal lane cannot be inferred from the public lane. This collapses the two hash calls in (C2) to one, removing one logical sponge relation.

Table 2: Implemented ZK-ACE Groth16 replay predicate constraint breakdown. C1, C2, and the merged C3+C4 are sponge absorptions over the width-3 BN254 Poseidon backend.

Constraint	Inputs	Sponge permutations	R1CS (measured)
(C1) Commitment consistency	3	2	480
(C2) Derivation correctness	4	3	720
(C3+C4) Authorization + replay	7	4	960
(C5) Domain sep. + <code>enforce_equal</code>	—	—	4
Total (implemented)		9	2,155

C3 + C4: two calls $\rightarrow$ single multi-output squeeze. Constraint (C3) computes $\text{Auth} = H(\text{REV} \parallel \text{Ctx} \parallel \text{TxHash} \parallel \text{domain} \parallel \text{nonce})$ and constraint (C4) derives rp_com as either $H(\text{Auth} \parallel \text{domain})$ (nullifier mode) or $H(\text{ID}_{\text{com}} \parallel \text{nonce})$ (nonce mode). The C4 call is a second invocation on the output of C3, again following HMAC-style defensive design. Since Auth already incorporates nonce and domain , both outputs can be obtained by squeezing two lanes from the same sponge state after a single absorption of $(\text{REV}, \text{Ctx}, \text{TxHash}, \text{domain}, \text{nonce})$. This collapses C3 and C4 into one hash call, removing a further logical sponge relation.

Optimized constraint summary. The measured constraint count is 2,155 R1CS (Groth16/BN254) and 341 active rows / 361 AIR constraint expressions (Circle STARK), as reported by the current implementation and benchmark suite. This optimization is implemented in the reference implementation. The security arguments of Section 8 carry over: the merged (C3+C4) sponge enforces the same binding relation over $(\text{REV}, \text{Ctx}, \text{TxHash}, \text{domain}, \text{nonce})$.

Each squeeze output is assigned an explicit semantic lane label to prevent cross-use: `SQUEEZE_TARGET` (C2, second lane), `SQUEEZE_AUTH` (C3+C4, first lane), and `SQUEEZE_RP` (C3+C4, second lane). Under the sponge indistinguishability framework [30, 31], these domain-separated outputs of the same sponge instance are modeled as independent random oracle outputs, provided the capacity is sufficiently large relative to the total squeezed output. Concretely, the Groth16 backend has capacity ≈ 254 bits and the STARK backend has capacity ≈ 248 bits. Whether these capacity sizes provide sufficient quantum collision resistance margins (BHT algorithm gives $\approx 2^{n/3}$ quantum queries for an n -bit hash) requires dedicated parameter analysis; the claim of adequacy is conditional on that analysis and should not be taken as established.

Aggressive 2-call variant. A further reduction absorbs (C2) into the same sponge call as (C3+C4), treating (REV, Ctx) derivation and $(\text{REV}, \text{Ctx}, \text{TxHash}, \text{domain}, \text{nonce})$ authorization as a single multi-rate absorption with three squeezed outputs: derived key commitment, authorization token, and replay-prevention commitment. This matches the 2-call lower bound identified by Elias [33]. While implementable, the merged absorption couples the security proofs for derivation correctness (C2) and authorization binding (C3) into a single reduction, making formal separation of these guarantees less direct. We present this as a future direction; formalizing the merged reduction is left as an open problem.

7.5 Replay Protection Designs

ZK-ACE supports multiple replay-prevention strategies. We present two canonical options.

Option A: Nonce registry (account-style model). In an account-style model, each identity commitment maintains a nonce state on-chain. The proof includes a nonce commitment (Eq. 6) as a public input, and the verifier enforces that the nonce is fresh according to the protocol rule (e.g., strictly increasing, or equal to the expected next nonce).

Trade-offs. Nonce registries provide simple, deterministic replay protection and integrate naturally with account abstraction. However, they introduce per-identity state updates and reveal ordering information, which may be undesirable in privacy-preserving settings. Additionally, nonce-based models require sequential submission, limiting parallelism.

Option B: Nullifier set (privacy-style model). In a privacy-oriented model, the proof outputs a nullifier derived from the authorization token (Eq. 7). The verifier maintains a global set of spent nullifiers and rejects any transaction whose nullifier has appeared before.

Trade-offs. Nullifiers support single-use authorization without requiring a per-identity nonce counter and can better accommodate unlinkability across authorizations. However, maintaining and checking a nullifier set introduces global state growth, and careful design is required to prevent unintended linkage across domains.

Remark 7.1. *The choice between nonce and nullifier models is deployment-specific and does not affect the core security properties of ZK-ACE. Both models satisfy the replay-resistance definition (Definition 8.2) under their respective verifier enforcement rules.*

8 Security Definitions and Analysis

This section formalizes the security properties of ZK-ACE via explicit game-based definitions and provides reduction-based proofs to standard assumptions, following the code-based game-playing methodology [24]. For a comprehensive treatment of the underlying cryptographic tools, we refer the reader to [25]. Throughout, $\mathcal{A}$ denotes a PPT adversary (classical probabilistic polynomial-time). The theorem statements therefore establish classical security; a quantum-adversary (QPT) interpretation requires that (i) the instantiated proof system achieves knowledge soundness against QPT adversaries, and (ii) the random oracle is programmable in the quantum setting (QROM). When the Circle STARK backend is used with Poseidon2 modeled as a quantum-accessible random oracle, the same theorem statements are conjectured to hold under QPT adversaries, conditional on the backend’s quantum knowledge soundness and the quantum collision-resistance margin of the hash function (see Section 13). Formalizing these conditions is beyond the scope of this work.

Remark 8.1 (Mapping practical attack scenarios to formal games). *The threat model (Section 3.5) grants $\mathcal{A}$ the ability to submit, reorder, replay, and substitute transaction payloads. The four security games defined below capture these capabilities as follows:*

- Unauthorized authorization (identity theft): *An adversary who does not possess REV attempts to produce a valid proof for an on-chain identity commitment. This is captured by the authorization soundness game (Definition 8.1).*
- Front-running and transaction substitution: *An adversary observes a valid authorization proof in the mempool and attempts to rebind it to a different transaction hash. Because the proof is cryptographically bound to TxHash via constraint (C3), this is captured by the substitution-resistance game (Definition 8.3).*
- Replay and state-race attacks: *An adversary attempts to resubmit a previously accepted authorization, or to exploit concurrent submissions to cause double-spending. Both are captured by the replay-resistance game (Definition 8.2), whose analysis covers the nonce (sequential ordering) and nullifier (set-membership) enforcement models.*
- Cross-chain or cross-application proof migration: *An adversary takes a valid proof from one domain and attempts to use it in another. This is captured by the cross-domain separation game (Definition 8.4), enforced by the inclusion of domain in all commitment and binding computations (C1–C5).*

Together, the four games provide coverage of the attack surface described in the threat model. Attacks not captured—such as compromise of the underlying REV or denial-of-service at the consensus layer—are out of scope, as noted in Section 3.5.

8.1 Authorization Soundness

Definition 8.1 (Authorization Soundness Game). *The game $\text{Game}_A^{\text{auth}}(\lambda)$ proceeds as follows:*

1. **Setup.** *The challenger generates a fresh identity: $\text{REV} \xleftarrow{\$} \{0,1\}^{256}$, $\text{salt} \xleftarrow{\$} \{0,1\}^\lambda$, and $\text{ID}_{\text{com}} = H(\text{REV} \parallel \text{salt} \parallel \text{domain})$. The verifier key vk and the commitment ID_{com} are given to $\mathcal{A}$. The values REV and salt are not disclosed.*
2. **Oracle access.** *$\mathcal{A}$ may observe all public on-chain data, including previously accepted proofs and their public inputs. $\mathcal{A}$ has no access to the sealed artifact SA , the credential Cred , or any Unseal oracle; $\mathcal{A}$ interacts with the DIDP only through the public oracles $\mathcal{O}_{\text{pub}}$ (which returns ID_{com} given $\text{salt}, \text{domain}$) and $\mathcal{O}_{\text{derive}}$ (which returns $H(\text{Derive}(\text{REV}, \text{Ctx}))$ for any queried context Ctx). This precisely matches the adversarial interface of the DIDP recovery game (Section 3.3).*
3. **Output.** *$\mathcal{A}$ outputs a candidate $(\pi^*, \pi_{\text{pub}}^*)$ where π_{pub}^* includes ID_{com} as the identity commitment.*
4. **Win condition.** *$\mathcal{A}$ wins if $\text{ZKVerify}(\text{vk}, \pi^*, \pi_{\text{pub}}^*) = 1$ and the replay predicate passes for the submitted tuple.*

The advantage of $\mathcal{A}$ is $\text{Adv}_A^{\text{auth}}(\lambda) = \Pr[\text{Game}_A^{\text{auth}}(\lambda) = 1]$.

Remark 8.2 (Strength of the adversarial model). *Definition 8.1 uses passive observation of accepted proofs as the baseline adversarial interface. This model is sufficient for our core reductions. Extending to an explicit adaptive chosen-message authorization oracle is possible, but requires additional proof-system-level assumptions (e.g., simulation soundness/non-malleability of the authorization interface) to preserve an unchanged bound. We leave that strengthening as future formalization.*

Theorem 8.1 (Authorization Soundness). *Under (i) knowledge soundness of the proof system, (ii) collision resistance of H , and (iii) identity-root recovery hardness of the DIDP (Section 3.3), for any PPT adversary $\mathcal{A}$:*

$$\text{Adv}_A^{\text{auth}}(\lambda) \leq \text{Adv}_{\mathcal{B}_1}^{\text{ks}}(\lambda) + \text{Adv}_{\mathcal{B}_2}^{\text{cr}}(\lambda) + \text{Adv}_{\mathcal{B}_3}^{\text{rec-didp}}(\lambda) + \text{negl}(\lambda).$$

Proof. Suppose $\mathcal{A}$ outputs $(\pi^*, \pi_{\text{pub}}^*)$ that the verifier accepts. We construct reductions $\mathcal{B}_1, \mathcal{B}_2, \mathcal{B}_3$ as follows.

Step 1: Witness extraction. By the knowledge-soundness property of the proof system, there exists a PPT extractor $\mathcal{E}$ such that, given $\mathcal{A}$'s accepting proof and randomness, $\mathcal{E}$ outputs a witness $w^* = (\text{REV}^*, \text{salt}^*, \text{Ctx}^*, \text{nonce}^*, \text{aux}^*)$ satisfying all circuit constraints (C1)–(C5), except with probability at most $\text{Adv}_{\mathcal{B}_1}^{\text{ks}}(\lambda)$. We construct $\mathcal{B}_1$ that simulates the game for $\mathcal{A}$ and, upon receiving an accepting proof, invokes $\mathcal{E}$ to recover the witness.

Step 2: Case analysis on the extracted witness. Assume extraction succeeds. By constraint (C1), the extracted witness satisfies:

$$H(\text{REV}^* \parallel \text{salt}^* \parallel \text{domain}) = \text{ID}_{\text{com}} = H(\text{REV} \parallel \text{salt} \parallel \text{domain}).$$

We consider two cases:

Case 1: $(\text{REV}^*, \text{salt}^*) \neq (\text{REV}, \text{salt})$. In this case, $\mathcal{A}$ has produced two distinct preimages of ID_{com} under H , constituting a collision. We construct $\mathcal{B}_2$ that embeds a collision-resistance challenge into H and uses $\mathcal{A}$ as a subroutine: $\mathcal{B}_2$ simulates the game honestly using the challenge hash,

and when extraction yields $(\text{REV}^*, \text{salt}^*) \neq (\text{REV}, \text{salt})$, outputs both preimages as a collision. Thus $\text{Adv}_{\mathcal{B}_2}^{\text{cr}}(\lambda) \geq \Pr[\text{Case 1}]$.

Case 2: $\text{REV}^* = \text{REV}$ and $\text{salt}^* = \text{salt}$. In this case, the extracted witness contains the correct REV. By the oracle-access restriction in Definition 8.1, $\mathcal{A}$ interacted with the DIDP only through $\mathcal{O}_{\text{pub}}$ and $\mathcal{O}_{\text{derive}}$, and had no access to SA, Cred, or Unseal. This environment is exactly the adversarial interface of the DIDP recovery game; therefore, $\mathcal{A}$ producing a proof from which REV can be extracted constitutes a win in the recovery game, breaching DIDP identity-root recovery hardness (Section 3.3). We construct $\mathcal{B}_3$ as follows. $\mathcal{B}_3$ participates in the recovery game $\text{Game}_{\text{DIDP}}^{\text{rec-didp}}$, receiving oracle access to $\mathcal{O}_{\text{pub}}$ and $\mathcal{O}_{\text{derive}}$ from the DIDP challenger. $\mathcal{B}_3$ samples $\text{salt} \xleftarrow{\$} \{0, 1\}^\lambda$ and queries $\mathcal{O}_{\text{pub}}(\text{salt}, \text{domain})$ to obtain $\text{ID}_{\text{com}} = H(\text{REV} \parallel \text{salt} \parallel \text{domain})$ without learning REV. Similarly, $\mathcal{B}_3$ queries $\mathcal{O}_{\text{derive}}(\text{Ctx})$ to obtain the target-binding value $H(\text{Derive}(\text{REV}, \text{Ctx}))$ for any context needed to simulate public on-chain data. Using ID_{com} and the derived public values, $\mathcal{B}_3$ faithfully simulates the authorization soundness game for $\mathcal{A}$. When extraction succeeds and yields $\text{REV}^* = \text{REV}$, $\mathcal{B}_3$ outputs REV^* as its answer in the DIDP recovery game. Thus $\text{Adv}_{\mathcal{B}_3}^{\text{rec-didp}}(\lambda) \geq \Pr[\text{Case 2}]$.

Step 3: Combining. By a union bound over all failure events:

$$\text{Adv}_{\mathcal{A}}^{\text{auth}}(\lambda) \leq \text{Adv}_{\mathcal{B}_1}^{\text{ks}}(\lambda) + \text{Adv}_{\mathcal{B}_2}^{\text{cr}}(\lambda) + \text{Adv}_{\mathcal{B}_3}^{\text{rec-didp}}(\lambda) + \text{negl}(\lambda). \quad \square$$

8.2 Replay Resistance

Definition 8.2 (Replay-Resistance Game). *The game $\text{Game}_{\mathcal{A}}^{\text{replay}}(\lambda)$ proceeds as follows:*

1. **Setup.** *As in Definition 8.1. The honest prover produces one valid authorization tuple $(\pi_1, \pi_{1,\text{pub}})$ for a transaction with hash TxHash_1 , which is accepted and applied to the verifier state (nonce incremented or nullifier inserted).*
2. **Oracle access.** *$\mathcal{A}$ receives $(\pi_1, \pi_{1,\text{pub}})$ and full access to on-chain state.*
3. **Win condition.** *$\mathcal{A}$ wins if it causes acceptance of a second authorization tuple $(\pi^*, \pi_{\text{pub}}^*)$ where π_{pub}^* includes ID_{com} and the same logical authorization event (i.e., the tuple violates the protocol's replay-prevention invariant).*

Theorem 8.2 (Replay Resistance). *Under the assumptions of Theorem 8.1 and correct verifier enforcement of the replay predicate, for any PPT adversary $\mathcal{A}$:*

$$\text{Adv}_{\mathcal{A}}^{\text{replay}}(\lambda) \leq \text{Adv}_{\mathcal{A}}^{\text{auth}}(\lambda) + \text{negl}(\lambda).$$

Proof. We consider each replay model:

Nonce model. As noted in Section 7.3 (C4), the nonce commitment $\text{rp_com} = H(\text{ID}_{\text{com}} \parallel \text{nonce})$ admits two sub-models depending on whether nonce is a public or private witness:

Sub-model A: public nonce. The nonce is included as a public input; the verifier reads its value directly and enforces strict monotonicity ($\text{nonce} = n + 1$ after an authorization at nonce n).

- *Direct replay:* Resubmitting $(\pi_1, \pi_{1,\text{pub}})$ presents nonce n . The verifier rejects because the expected nonce is $n + 1$.
- *New proof with stale nonce:* Any new proof exposing nonce $\leq n$ is rejected by the monotonicity check.
- *New proof with fresh nonce:* Producing a valid proof for nonce $n + 1$ requires a valid witness satisfying (C1)–(C5), which reduces to the authorization soundness game (Theorem 8.1).

Sub-model B: opaque nonce commitment. The nonce is a private witness; the chain observes only rp_com . Because nonce is not visible on-chain, strict monotonicity cannot be enforced. The verifier instead treats rp_com as a single-use token and rejects any rp_com that has appeared before, reducing to nullifier-set semantics (Sub-model B is analyzed under the Nullifier model below).

Nullifier model. After the honest authorization, the nullifier $\text{rp_com}_1 = H(\text{Auth}_1 \parallel \text{domain})$ is inserted into the nullifier set $\mathcal{N}$.

- *Direct replay:* Resubmitting $(\pi_1, \pi_{1,\text{pub}})$ produces the same $\text{rp_com}_1 \in \mathcal{N}$, which is rejected.
- *Same authorization, different proof:* Any proof for the same $(\text{REV}, \text{Ctx}, \text{TxHash}, \text{domain}, \text{nonce})$ produces the same Auth_1 and hence the same nullifier, which is rejected.
- *Different authorization parameters:* Changing any component of the Auth computation yields a different Auth' and a fresh nullifier $\text{rp_com}' \notin \mathcal{N}$. This represents a genuinely new authorization event (different transaction, context, or nonce) and is accepted—correctly, as it is not a replay.

In both models, the only way for $\mathcal{A}$ to cause acceptance of a replayed authorization event is to produce a valid proof with a fresh replay token, which requires knowledge of a valid witness. This reduces to authorization soundness. $\square$

8.3 Substitution Resistance

Definition 8.3 (Substitution-Resistance Game). *The game $\text{Game}_{\mathcal{A}}^{\text{subst}}(\lambda)$ proceeds as follows:*

1. **Setup.** *As in Definition 8.1. An honest authorization tuple (π, π_{pub}) is generated for public inputs $(\text{ID}_{\text{com}}, \text{TxHash}, \text{domain}, \text{target}, \text{rp_com})$.*
2. **Oracle access.** *$\mathcal{A}$ receives (π, π_{pub}) .*
3. **Win condition.** *$\mathcal{A}$ wins if the verifier accepts (π, π'_{pub}) where $\pi'_{\text{pub}} \neq \pi_{\text{pub}}$, i.e., the same proof is accepted with modified public inputs (different TxHash , target , or domain).*

Theorem 8.3 (Substitution Resistance). *Under the public-input binding property of the proof system, for any PPT adversary $\mathcal{A}$:*

$$\text{Adv}_{\mathcal{A}}^{\text{subst}}(\lambda) \leq \text{Adv}_{\mathcal{B}}^{\text{pib}}(\lambda) + \text{negl}(\lambda).$$

Proof. Suppose $\mathcal{A}$ outputs $\pi'_{\text{pub}} \neq \pi_{\text{pub}}$ such that $\text{ZKVerify}(\text{vk}, \pi, \pi'_{\text{pub}}) = 1$. Then the same proof π verifies under two distinct public input tuples. We construct $\mathcal{B}$ that embeds this as a public-input binding challenge: $\mathcal{B}$ simulates the substitution game for $\mathcal{A}$, and upon $\mathcal{A}$'s success, outputs $(\pi, \pi_{\text{pub}}, \pi'_{\text{pub}})$ as a violation of public-input binding.

By the public-input binding property (Section 3.4), this occurs with at most negligible probability, giving:

$$\text{Adv}_{\mathcal{A}}^{\text{subst}}(\lambda) \leq \text{Adv}_{\mathcal{B}}^{\text{pib}}(\lambda) + \text{negl}(\lambda). \quad \square$$

8.4 Cross-Domain Separation

Definition 8.4 (Cross-Domain Separation Game). *The game $\text{Game}_{\mathcal{A}}^{\text{domain}}(\lambda)$ proceeds as follows:*

1. **Setup.** *The challenger registers the same identity on two distinct domains $\text{domain}_1 \neq \text{domain}_2$ with respective commitments $\text{ID}_{\text{com}}^{(1)} = H(\text{REV} \parallel \text{salt}_1 \parallel \text{domain}_1)$ and $\text{ID}_{\text{com}}^{(2)} = H(\text{REV} \parallel \text{salt}_2 \parallel \text{domain}_2)$, where $\text{salt}_1, \text{salt}_2$ are independently sampled. An honest authorization proof (π, π_{pub}) is generated for domain domain_1 .*
2. **Win condition.** *$\mathcal{A}$ wins if it causes acceptance of an authorization tuple on domain domain_2 using material from the domain- domain_1 authorization (either by proof reuse or derivation).*

Theorem 8.4 (Cross-Domain Separation). *Under collision resistance of H and public-input binding of the proof system, for any PPT adversary $\mathcal{A}$:*

$$\text{Adv}_{\mathcal{A}}^{\text{domain}}(\lambda) \leq \text{Adv}_{\mathcal{B}_1}^{\text{pib}}(\lambda) + \text{Adv}_{\mathcal{B}_2}^{\text{cr}}(\lambda) + \text{negl}(\lambda).$$

Proof. We consider two attack strategies:

Proof reuse. If $\mathcal{A}$ submits the original proof π (generated for domain domain_1) to the domain- domain_2 verifier, the public inputs must include $\text{ID}_{\text{com}}^{(2)}$ and domain_2 . Since π was generated for

public inputs containing $ID_{\text{com}}^{(1)}$ and domain_1 , and $(ID_{\text{com}}^{(1)}, \text{domain}_1) \neq (ID_{\text{com}}^{(2)}, \text{domain}_2)$, verification fails by public-input binding.

Commitment collision. If $\mathcal{A}$ attempts to find $(\text{REV}', \text{salt}', \text{domain}_2)$ such that $H(\text{REV}' \parallel \text{salt}' \parallel \text{domain}_2) = ID_{\text{com}}^{(1)}$, this yields two distinct domain-tagged inputs mapping to the same hash value and therefore constitutes a collision event under H .

Moreover, by constraint (C5), domain is included in all circuit bindings. A proof generated with $\text{domain} = \text{domain}_1$ cannot satisfy the verifier's constraint checks when the declared domain is domain_2 , as the commitment, authorization token, and replay-prevention commitment all incorporate domain . Combining: $\text{Adv}_{\mathcal{A}}^{\text{domain}}(\lambda) \leq \text{Adv}_{\mathcal{B}_1}^{\text{piib}}(\lambda) + \text{Adv}_{\mathcal{B}_2}^{\text{cr}}(\lambda) + \text{negl}(\lambda)$. $\square$

8.5 Composition

Corollary 8.1 (Combined Security). *Under the assumptions of Theorems 8.1–8.4, ZK-ACE simultaneously satisfies authorization soundness, replay resistance, substitution resistance, and cross-domain separation against PPT adversaries. Specifically, for any PPT adversary $\mathcal{A}$ attacking any single property, there exist PPT reductions $\mathcal{B}_1, \dots, \mathcal{B}_4$ such that:*

$$\text{Adv}_{\mathcal{A}}^{\text{combined}}(\lambda) \leq \text{Adv}_{\mathcal{B}_1}^{\text{ks}}(\lambda) + 2 \cdot \text{Adv}_{\mathcal{B}_2}^{\text{cr}}(\lambda) + \text{Adv}_{\mathcal{B}_3}^{\text{rec-didp}}(\lambda) + 2 \cdot \text{Adv}_{\mathcal{B}_4}^{\text{piib}}(\lambda) + \text{negl}(\lambda),$$

where the factor of 2 on Adv^{cr} arises because both the authorization soundness proof (Theorem 8.1, Case 1) and the cross-domain separation proof (Theorem 8.4) each reduce to collision resistance, and the factor of 2 on Adv^{piib} arises because both the substitution resistance proof (Theorem 8.3) and the cross-domain separation proof each reduce to public-input binding. The bound is negligible under the stated assumptions.

Proposition 8.1 (Completeness). *If the honest prover holds a valid witness $w = (\text{REV}, \text{salt}, \text{Ctx}, \text{nonce}, \text{aux})$ satisfying all constraints (C1)–(C5), and the replay-prevention state permits the submitted token (i.e., nonce is fresh or nullifier is unspent), then the verifier accepts the resulting proof with probability $1 - \text{negl}(\lambda)$.*

Proof. By the completeness property of the underlying proof system, an honestly generated proof for a satisfying witness is accepted with overwhelming probability. The replay-prevention predicate passes by hypothesis on the freshness of the submitted nonce or nullifier. All public inputs are computed consistently with the witness by construction, so the verifier's constraint checks are satisfied. $\square$

Remark 8.3 (Privacy Guarantee). *By the zero-knowledge property of the underlying proof system, ZK-ACE authorization proofs reveal no information about the private witness $(\text{REV}, \text{salt}, \text{Ctx}, \text{nonce}, \text{aux})$ beyond what is implied by the public inputs $(ID_{\text{com}}, \text{TxHash}, \text{domain}, \text{target}, \text{rp_com})$. In particular:*

- The 256-bit Root Entropy Value REV remains hidden. Since REV is the sole identity root from which all cryptographic material is derived, its concealment ensures that no derived keys can be computed by observers.
- The commitment salt salt is not disclosed, preserving identity unlinkability across commitments derived with different salts.
- The derivation context Ctx and nonce nonce are hidden, preventing observers from inferring the target algorithm, application domain, derivation index, or replay-prevention state beyond what the public rp_com reveals.

The information revealed to observers is limited to: (i) which identity commitment was used, (ii) which transaction was authorized, (iii) the domain and target context, and (iv) the replay-prevention token. This leakage profile is inherent to any authorization system that must bind proofs to specific transactions and enforce replay prevention, and represents the minimal information disclosure required for consensus verification.

9 On-Chain Verification and Deployment

9.1 Verification Interface

At the consensus layer, ZK-ACE exposes a minimal verification interface that accepts a zero-knowledge proof together with a fixed set of public inputs. Conceptually, the verifier evaluates a predicate of the form:

$$\text{ZKVerify}(\text{proof}, \text{public_inputs}) \in \{\text{accept}, \text{reject}\}.$$

Crucially, the verifier does not accept the prover’s declared public inputs at face value. Before evaluating the proof, the verifier must independently verify context consistency (Algorithm 1, steps 6–7): (i) recompute TxHash from the submitted transaction payload and confirm it matches the value declared in π_{pub} ; (ii) confirm that ID_{com} corresponds to a registered on-chain identity commitment; and (iii) confirm that **domain** and **target** are consistent with the expected transaction context. Only after these checks pass does the verifier evaluate the zero-knowledge proof and, upon successful verification, perform any required state updates associated with replay prevention:

- recording the consumed nonce in a per-identity nonce registry;
- inserting the nullifier into a global nullifier set to prevent reuse; or
- updating an authorization index or similar monotonic state variable.

These updates are deterministic and verifiable by all consensus participants. No identity root material (REV) or derived keys are ever written to chain state.

9.2 Compatibility with Account Abstraction

ZK-ACE is naturally compatible with account-abstraction (AA) architectures, including designs aligned with ERC-4337-style transaction flows [17].

Validator module integration. In an AA setting, ZK-ACE can be implemented as a *validator module* associated with an abstract account. Instead of checking a classical or post-quantum signature, the account validation logic invokes the ZK-ACE verification interface to determine whether the transaction is authorized by the committed identity.

Bundler and submission flow. The generation of zero-knowledge proofs occurs off-chain and is performed by the entity controlling the identity, typically the user’s wallet software. A bundler or relayer may collect user operations and submit them to the chain, but does not need access to any secret identity material. The bundler merely transports the proof and public inputs and has no influence over authorization correctness. This separation preserves the trust-minimized design of account abstraction while allowing proof generation and transaction submission responsibilities to be flexibly allocated.

9.3 Instantiation Flexibility

Proof system selection. ZK-ACE may be instantiated using different classes of zero-knowledge proof systems. SNARK-based systems (e.g., Groth16 [9]) offer the smallest proof sizes and fastest verification, at the cost of a trusted setup. Universal systems (e.g., PLONK [8]) avoid per-circuit trusted setup. Transparent systems (e.g., STARKs [10]) require no trusted setup at all but produce larger proofs. IPA-based systems (e.g., Bulletproofs [11]) offer a different trade-off between proof size and verification time. The framework does not rely on properties unique to any particular system beyond soundness, zero-knowledge, and public-input binding.

Deployment mode classification. ZK-ACE supports three distinct on-chain deployment configurations, which differ in proof system and the role of off-chain aggregation:

- **Groth16 (classical, direct on-chain).** 128-byte compressed proofs are submitted per transaction without aggregation. Total per-transaction on-chain data is 288 B (128 B proof + 160 B public inputs). This mode offers the smallest proof footprint but is not post-quantum secure; it is the appropriate choice for EVM-native deployments that do not require quantum resistance.
- **STARK individual (post-quantum-oriented, off-chain only).** Individual STARK proofs (≈ 107 KB) carry post-quantum-oriented security conditional on the instantiated hash and FRI parameters, but are far too large for per-transaction on-chain submission. This configuration is suited to off-chain audit trails or local verification workflows, not to direct on-chain use.
- **STARK aggregated (post-quantum-oriented, target deployment form).** Individual proofs are verified off-chain by the block builder, which submits one aggregate proof per block. Per-transaction consensus-visible data is ≈ 160 B (five public input fields) plus an amortized share of the block-level aggregate proof. This is the intended production deployment mode for post-quantum-ready systems.

The structural data accounting in Section 10 and the benchmarks in Section 12 are organized around this three-mode classification. Post-quantum security claims for the STARK modes are conditional on the parameter choices discussed in Section 3.4.

Deployment-oriented instantiation. A deployment-oriented ZK-ACE configuration can instantiate the authorization circuit with the Circle STARK backend (Stwo [27]) benchmarked in Section 12. This choice is motivated by three considerations: (i) STARKs require no trusted setup, eliminating a significant operational and trust burden; (ii) STARK security relies on hash-based assumptions, making it a natural fit for post-quantum-oriented deployments when parameters provide adequate quantum security margins [32]; and (iii) the transparent setup permits permissionless deployment without a multi-party computation ceremony. The trade-off is larger proof sizes (≈ 107 KB vs. 128 B for Groth16), which are too large for per-transaction on-chain submission and motivate the aggregation and builder-off-path models discussed below. The Groth16 benchmarks in Section 12.2 remain valid as a lower bound on proof-system performance and serve to demonstrate circuit feasibility at minimal constraint count.

Hash function parameterization. ZK-friendly hash functions, such as Poseidon [7], are used within the circuit for commitments and bindings. Parameter choices (e.g., arity, round numbers, field size) are left to the deployment context. The security arguments of ZK-ACE rely only on standard collision and preimage resistance assumptions, not on any specific parameter set.

Proof-system migration. Because identity commitments are proof-system agnostic (they depend only on the hash function, not the ZK proof system), verifier and prover upgrades do not require identity rotation. A chain can migrate from one proof system to another without invalidating existing identity commitments.

10 Structural On-Chain Data Accounting

This section provides a structural accounting of on-chain authorization data under different authorization models. The purpose is to compare the *protocol-imposed data artifacts* required by post-quantum signature-based authorization and by ZK-ACE authorization, independent of concrete implementations or performance benchmarks.

Table 3: Post-quantum signature artifact sizes for standardized schemes.

Scheme	NIST Level	Signature Size	Public Key
ML-DSA-44 [3]	2	2,420 bytes	1,312 bytes
ML-DSA-65 [3]	3	3,309 bytes	1,952 bytes
ML-DSA-87 [3]	5	4,627 bytes	2,592 bytes
SLH-DSA-128f [4]	1	17,088 bytes	32 bytes
FN-DSA-512 [5]	1	$\approx$ 666 bytes	897 bytes
Ed25519 (classical)		64 bytes	32 bytes
secp256k1 ECDSA (classical)		$\approx$ 71 bytes	33 bytes

10.1 Accounting Methodology

The analysis is not based on experimental measurements, runtime benchmarks, or gas profiling. Instead, it relies on protocol-level accounting of the data objects that must be included on-chain for transaction authorization under each model. Specifically, we compare:

- the mandatory authorization artifacts that consensus participants must receive, verify, and store; and
- the asymptotic size of these artifacts as dictated by cryptographic standards and protocol structure.

10.2 Baseline: Post-Quantum Signature Artifacts

In a post-quantum signature-based authorization model, each transaction must carry a cryptographic signature object that is verified by the consensus layer. For lattice-based signature schemes, standardized parameter sets result in signature sizes on the order of several kilobytes. Table 3 summarizes representative sizes.

The defining characteristic is that post-quantum signature objects are *kilobyte-scale*. These objects must be transmitted, verified, and permanently recorded as part of the ledger, making their size a first-order contributor to on-chain data growth.

10.3 ZK-ACE Authorization Artifacts

Under ZK-ACE, explicit post-quantum signature objects are not placed on-chain. Instead, authorization is represented by a combination of compact commitments and succinct zero-knowledge proofs. The required on-chain artifacts per transaction are:

- **Identity commitment:** A hash-based commitment to the deterministic identity (ID_{com}), typically occupying 32 bytes.
- **Zero-knowledge proof:** A succinct proof attesting to authorization correctness. The Groth16 backend produces 128-byte compressed proofs. The STARK backend produces $\approx$ 107 KB individual proofs, which are aggregated per-block by the builder; individual STARK proofs never appear on-chain.
- **Public inputs:** A small set of consensus-visible values, including TxHash, domain, target, and rp_{com} , contributing approximately 160 bytes (5×32 B).

Crucially, none of these artifacts scale with the size or internal structure of post-quantum signature schemes.

10.4 Order-of-Magnitude Comparison

Comparing the structural requirements of the two models yields a clear order-of-magnitude difference in on-chain authorization data, summarized in Table 4:

Table 4: Structural comparison of per-transaction authorization data. PQC column assumes first-use public-key transmission. STARK column assumes mandatory per-block aggregation: the 160 B figure covers public inputs only; amortized aggregation proof overhead is not included. Groth16 column uses measured compressed proof size. Neither column accounts for chain-specific encoding overhead.

Component	PQC Sig Model	ZK-ACE (STARK)	ZK-ACE (Groth16)
Signature / Proof	2,420–4,627 B	≈ 0 B (amortized)	128 B
Public key (amortized)	1,312–2,592 B	0 B	0 B
Identity commitment	—	32 B (reused)	32 B (reused)
Public inputs	—	160 B	160 B
Total per TX	3,732–7,219 B	≈ 160 B	288 B

Under the STARK aggregation model, the structural replacement implies a *per-transaction on-chain data reduction of 23–45 $\times$* (≈ 160 B vs. 3,732–7,219 B). Under the Groth16 model, the reduction is 13–25 $\times$ (288 B vs. 3,732–7,219 B). The detailed per-scheme compression ratios are given in Table 9 (Section 12.2). Importantly, this reduction does not arise from compressing post-quantum signatures. Rather, it is achieved by *eliminating the need to place post-quantum signature material on-chain* and replacing it with succinct proofs of authorization semantics.

Comparison caveats. Table 4 presents a structural, protocol-level comparison intended to illustrate the order-of-magnitude difference in authorization artifact sizes. Several factors affect the precise ratio in any concrete deployment: (i) public-key amortization depends on whether the sender’s key is already known to the chain (repeat senders amortize to zero in the PQC model, narrowing the gap); (ii) chain-specific encoding formats (RLP, SSZ, ABI encoding) add overhead to both models; and (iii) the STARK aggregation model assumes a block builder that aggregates individual proofs, adding builder-side computation. The comparison is therefore best understood as a structural accounting of the *cryptographic artifact sizes* mandated by each authorization model, rather than as a prediction of concrete gas costs or byte counts on any specific chain.

11 Scalability Extensions

11.1 Batch Authorization

Multiple independent authorization statements from distinct identities can be combined using batched proving or, in future deployments, proof-aggregation techniques. The current reference implementation benchmarks a tiled STARK batch path whose verification time grows with batch size; recursive aggregation would be required for constant-size verification artifacts over arbitrarily many authorizations.

Batch authorization preserves the individual security guarantees of each component statement: knowledge soundness of the aggregated proof implies knowledge soundness of each constituent statement. The aggregator (which collects individual proofs and produces the batch proof) is untrusted and does not learn any private witness material.

11.2 Recursive Proof Composition

Recursive composition can allow a top-level proof to attest to the validity of many base-level proofs [12, 13]. Applied to ZK-ACE, a recursive proof would verify k individual authorization

proofs and produce a single constant-size proof regardless of k . This is particularly useful in rollup architectures, but it is not what the current tiled-batch benchmark measures.

Such recursion would preserve all ZK-ACE security properties if the recursive proof system is knowledge-sound: the existence of an accepting recursive proof would imply the existence of valid witnesses for all constituent authorizations. Cross-domain separation is maintained because domain tags propagate through the recursion.

12 Reference Implementation and Benchmarks

To validate the structural analysis of Sections 7.3 and 10, we provide a reference implementation of the ZK-ACE circuit with two pluggable backends, and report Criterion.rs microbenchmarks [29].

12.1 Implementation

The reference implementation provides a pluggable backend architecture, allowing the same five-constraint circuit semantics (C1)–(C5) to be proven under fundamentally different proof systems. A compile-time feature flag selects between:

- **Circle STARK backend** (default, transparent hash-based backend). Built on Stwo [27], this backend operates over the Mersenne-31 (M31) field using Poseidon2 hash (width $t = 16$, $\alpha = 5$) and the FRI protocol. Verification relies only on hash functions—no elliptic curve assumptions—making it post-quantum-oriented under adequate hash and FRI parameters. The setup is transparent: no trusted ceremony is required. The circuit produces 341 active rows (11 Poseidon2 permutations, $\log_size=9$), 100 main trace columns, 34 preprocessed columns, and 361 AIR constraint expressions.
- **Groth16/BN254 backend** (compact classical proofs). Built on arkworks [28], this backend uses a manual Poseidon implementation (width $t = 3$, $\alpha = 5$, 8 full rounds, 56 partial rounds) over the BN254 scalar field with Groth16 [9] proving. It produces the smallest possible proofs (128 bytes) but relies on elliptic curve assumptions that are not quantum-resistant. The circuit compiles to 2,155 R1CS constraints.

Both backends produce identical constraint counts across the nonce-registry and nullifier-set replay modes, confirming that the replay-mode selection does not affect circuit complexity. Each backend includes a native (off-chain) hash implementation with identical parameters, ensuring functional equivalence between the in-circuit computation and the off-chain witness generation.

12.2 Benchmarks

All measurements were collected using Criterion.rs (100 samples per benchmark) on Apple Silicon (M-series), single-threaded execution, release-mode compiler optimizations (`--release`). These figures should be interpreted as reference measurements for the reported implementation and hardware; a public artifact should identify the repository URL, commit hash, Rust toolchain, benchmark commands, and machine configuration used to reproduce them.

Circle STARK backend. Table 5 reports the core STARK operations for a single authorization proof. The STARK backend is faster than Groth16 in both proving and verification. This follows from the underlying field arithmetic: Mersenne-31 operations (31-bit) are an order of magnitude cheaper than BN254 scalar field operations (254-bit), and STARK proving requires no elliptic curve multi-scalar multiplications.

Groth16/BN254 backend. Table 6 reports the Groth16 operations.

Table 5: Circle STARK (Stwo) benchmarks for ZK-ACE (single-threaded, Apple Silicon, Criterion.rs medians). The NonceRegistry and NullifierSet semantics share the same circuit after squeeze2 unification, so this table reports the unified replay-circuit path.

Operation	Unified path
Prove (per transaction)	14.5 ms
Verify (per transaction)	1.1 ms
Proof size	≈ 107 KB

Table 6: Groth16/BN254 benchmarks for ZK-ACE (single-threaded, Apple Silicon, Criterion.rs medians). The two replay semantics share the same circuit after squeeze2 unification, so this table reports the unified replay-circuit path.

Operation	Unified path
Prove (per transaction)	37.3 ms
Verify (per transaction)	1.6 ms
Proof size	128 B

Batch STARK proving. Table 7 reports batch proving and verification timings for the Circle STARK backend across batch sizes 1–16. Batch prove time scales near-linearly with N because trace generation dominates and is $O(N)$; the FRI commitment layer adds a $O(\log N)$ overhead that is small in practice. Batch verification provides moderate per-transaction amortization in the reference tiled-trace prototype, but verification time still grows with batch size. In the current measurements, batch verification rises from 3.18 ms at $N = 1$ to 37.97 ms at $N = 16$, or about 2.37 ms per transaction. Note that single-transaction non-batch verification (Table 5, 1.1 ms) is faster than batch $N = 1$ verify (3.18 ms) because the batch code path embeds per-transaction public input values into preprocessed columns that must be recomputed on each verify call.

Backend comparison. Table 8 summarizes the key differences between the two backends.

Verification latencies of 1.1–1.6 ms are well within budget for both L1 and rollup settings. For comparison, a secp256k1 ECDSA verification on the same hardware takes approximately 50–100 μ s; the ZK-ACE overhead is roughly 10–15 $\times$ that of a classical signature check, but replaces what would otherwise be a multi-kilobyte post-quantum signature.

Proof size and on-chain model. The Groth16 backend produces 128-byte compressed proofs (three point-compressed BN254 group elements), yielding 288 B total authorization data per transaction (128 B proof + 160 B public inputs).

The STARK backend produces individual proofs of ≈ 107 KB, which are too large for per-transaction on-chain submission. Under the mandatory aggregation architecture (Section 11.1), individual STARK proofs are verified off-chain by the block builder, who produces a single aggregate proof per block in deployments that add recursive aggregation. Per-transaction consensus-visible data consists of the five public input fields—approximately **160 bytes**—plus an amortized share of the aggregated proof submitted once per block. Under a builder-off-path model, aggregate proof verification is not performed by chain consensus, so the per-transaction consensus footprint remains ≈ 160 bytes; the aggregation cost falls entirely on the block builder.

Compression ratios versus post-quantum signatures. Table 9 reports the per-transaction authorization data reduction under both on-chain models.

Table 7: Circle STARK batch prove/verify timings and proof sizes (single-threaded, Apple Silicon, Criterion.rs medians, zk-ace v0.4.0). Proof size is the serialized ACBP batch proof; amortized size divides by N .

N	log	Prove	Prove/tx	Verify	Verify/tx	Proof	Proof/tx
1	9	14.29 ms	14.29 ms	3.18 ms	3.18 ms	122 KB	122 KB
2	10	28.89 ms	14.45 ms	5.27 ms	2.64 ms	139 KB	69 KB
4	11	57.84 ms	14.46 ms	9.83 ms	2.46 ms	156 KB	39 KB
8	12	126.10 ms	15.76 ms	19.90 ms	2.49 ms	185 KB	23 KB
16	13	236.39 ms	14.77 ms	37.97 ms	2.37 ms	206 KB	13 KB

Table 8: Backend comparison for ZK-ACE reference implementation.

Aspect	Circle STARK (Stwo)	Groth16/BN254
Field	Mersenne-31 (M31)	BN254 Fr (≈ 254 -bit)
Hash	Poseidon2 ($t=16, \alpha=5$)	Poseidon ($t=3, \alpha=5$)
Proof system	FRI + AIR	R1CS + pairing
Constraints	341 active rows / 361 AIR expr.	2,155 R1CS
Prove time	14.5 ms	37.3 ms
Verify time	1.1 ms	1.6 ms
Proof size	≈ 107 KB	128 B
PQ-oriented	Conditional	No
Setup	Transparent	Trusted (MPC)

Under the STARK aggregation model, ZK-ACE achieves a **23–107 $\times$** reduction in per-transaction on-chain authorization data relative to direct post-quantum signatures, with transparent setup and post-quantum-oriented security conditional on the instantiated parameters. The Groth16 backend provides a **5–59 $\times$** reduction with minimal proof size but without post-quantum guarantees.

12.3 Comparative Constraint Analysis

Table 10 compares the circuit complexity of ZK-ACE against the estimated constraint counts required for in-circuit verification of post-quantum signatures, providing a quantitative basis for the reported 500–2,300 $\times$ constraint reduction.

The ZK-ACE circuit achieves its low constraint count by a fundamentally different strategy: rather than embedding signature verification logic (which requires non-native field emulation, NTTs, or hash chains), it proves a semantic authorization property via a small number of ZK-friendly hash evaluations. This **roughly 500–2,300 $\times$** constraint reduction is the primary quantitative result supporting the practical viability of identity-bound authorization as an alternative to ZK-compressed signature verification.

End-to-end authorization cost. Table 11 summarizes the end-to-end cost profile for a single ZK-ACE authorization under each backend.

Attestation and pipeline operations. Table 12 reports timings for the identity-layer primitives and the consensus-critical pipeline stages, measured on the ACE runtime using `cargo bench -p ace-runtime --features test-utils --bench pipeline_bench`. **Note:** Phase 2 (proof generation) uses a deterministic mock prover that simulates per-transaction SHA-256

Table 9: Authorization data per transaction: PQC signature vs. ZK-ACE. STARK column assumes mandatory per-block aggregation (≈ 160 B public inputs only). Groth16 column uses compressed proof (288 B total).

Scheme	Sig + PK	STARK	Ratio	Groth16	Ratio
ML-DSA-44 (L2)	3,732 B	160 B	$23.3\times$	288 B	$13.0\times$
ML-DSA-65 (L3)	5,261 B	160 B	$32.9\times$	288 B	$18.3\times$
ML-DSA-87 (L5)	7,219 B	160 B	$45.1\times$	288 B	$25.1\times$
SLH-DSA-128f	17,120 B	160 B	$107\times$	288 B	$59.4\times$
FN-DSA-512	1,563 B	160 B	$9.8\times$	288 B	$5.4\times$

Table 10: Circuit constraint comparison: ZK-ACE identity-bound authorization vs. in-circuit post-quantum signature verification. The ML-DSA and FN-DSA estimates are structural lower bounds based on the algebraic operations required for verification (NTTs, non-native modular arithmetic, range checks); actual implementations may exceed these bounds due to range-check gadgets, carry propagation, and conditional logic.

Circuit	Constraints	Dominant cost
ZK-ACE STARK (this work)	341 active rows / 361 AIR expr.	11 Poseidon2 permutations
ZK-ACE Groth16 (this work)	2,155 R1CS	9 Poseidon sponge permutations
In-circuit ML-DSA-44 verify	$\gtrsim 2\text{M}^\dagger$	4×4 NTTs + mod. arith.
In-circuit ML-DSA-65 verify	$\gtrsim 4\text{M}^\dagger$	6×5 NTTs + mod. arith.
In-circuit FN-DSA-512 verify	$\gtrsim 1\text{M}^\dagger$	FFT + Gram-Schmidt + mod. arith.
In-circuit SLH-DSA verify	$\gtrsim 5\text{M}^{\dagger\dagger}$	WOTS ⁺ chains + Merkle trees
In-circuit ECDSA verify [19]	$\approx 1.5\text{M}$	scalar mul. + mod. inverse

[†] Lower bound: $k \times \ell$ degree-256 NTTs in $\mathbb{Z}_q$ ($q = 2^{23} - 2^{13} + 1$) emulated over a ~ 254 -bit field.

^{††} Dominated by hash-chain evaluations for WOTS⁺ one-time signatures and Merkle authentication paths.

hashing and tree aggregation; it does *not* invoke real Groth16 proving. The mock-prover timings reflect the overhead of the proving *orchestration*—witness marshalling, tree construction, and data serialization—exclusive of the ZK proof computation itself, which is benchmarked separately in Table 6.

At batch size 2,000, the consensus-critical Attest-Execute path (Phase 1a + 1b + Merkle root) completes in ≈ 0.92 ms, well under 1% of a 400 ms slot window. The attestation-check phase processes each transaction in ≈ 124 ns, consistent with the sub-microsecond HMAC-SHA256 verification expected from the design. The end-to-end pipeline (with mock proving) takes ≈ 7.56 ms; replacing the mock prover with real ZK proving would add ≈ 14.5 ms (STARK) or ≈ 37.3 ms (Groth16) per transaction of prover work, which is parallelizable and off the critical consensus path (Section 11.1).

13 Limitations and Future Work

- **DIDP dependency.** The security of ZK-ACE is conditional on the assumed security properties of the underlying DIDP (Section 3.3). These properties—determinism, context isolation, and identity-root recovery hardness—follow from standard assumptions (collision resistance and high min-entropy of REV) when the derivation function is modeled as a random oracle. ACE-GF [2] is one concrete instantiation; independent analysis of any candidate DIDP would further strengthen the foundation of this work.
- **Nullifier-set state growth.** In the nullifier model, the global nullifier set grows mono-

Table 11: End-to-end single-authorization cost profile for ZK-ACE.

Metric	STARK (Stwo)	Groth16/BN254
Constraints	341 active rows / 361 AIR expr.	2,155 R1CS
Prove time	14.5 ms	37.3 ms
Verify time	1.1 ms	1.6 ms
Proof size	≈ 107 KB	128 B
Per-TX on-chain data	≈ 160 B (aggregated)	288 B
PQ-oriented	Conditional	No
Setup	Transparent	Trusted

Table 12: ACE runtime pipeline benchmarks (single-threaded, Apple M3 Pro). Phase 2 uses a mock prover; real Groth16 timings are in Table 6.

Operation	Batch size	Median time
HKDF key derivation (single)	1	1.16 μ s
Identity commitment (ID_{com})	1	270 ns
Attestation generate	1	2.25 μ s
Attestation verify	1	0.82 μ s
Phase 1a: AttestCheck	2,000	247 μ s (124 ns/tx)
Phase 1b: Execute	2,000	134 μ s (67 ns/tx)
State root (Merkle)	2,000	537 μ s
Block build	2,000	1.97 ms
Phase 2: mock prove [†]	2,000	4.16 ms
End-to-end pipeline [†]	2,000	7.56 ms
REV Argon2id seal/unseal	1	3.5 ms

[†]Mock prover: orchestration overhead only, not real ZK proof generation.

tonically with the number of authorization events. Long-term state management strategies (e.g., epoch-based pruning, Merkle-tree compaction) are an open engineering problem.

- **Single-platform benchmarks.** The benchmarks in Section 12 are collected on Apple Silicon (single-threaded CPU). Gas-precise, chain-specific cost estimates and GPU-accelerated proving times remain to be measured. Cross-platform validation (e.g., x86, ARM server, WASM) and formal circuit audits are deferred to follow-up work.
- **STARK proof size.** Individual STARK proofs (≈ 107 KB) require mandatory per-block aggregation for direct on-chain use; recursive aggregation overhead on the block builder has not been benchmarked.
- **Batch verification model.** The current STARK batch implementation uses tiled traces rather than recursive aggregation. Verification time grows with batch size in the benchmarked prototype; constant-verification aggregation remains future work.
- **Groth16 trusted setup.** The Groth16 backend requires a per-circuit trusted setup. The reference implementation uses a deterministic seed; production deployments must use MPC ceremonies.
- **Identity revocation and rotation.** ZK-ACE as specified does not include an on-chain mechanism for identity revocation or commitment rotation. Extending the framework with revocation semantics compatible with the identity-commitment model is a direction for future work.
- **Poseidon quantum security.** The post-quantum security of the Circle STARK backend rests on the assumption that Poseidon2 is collision-resistant against quantum adversaries.

Quantum collision attacks are more aggressively degraded than preimage attacks: Grover’s algorithm gives a $2^{n/2}$ quantum preimage bound, but the BHT algorithm [1] achieves collision finding in $\approx 2^{n/3}$ quantum queries. Consequently, a 248-bit capacity Poseidon2 over M31 provides only ≈ 83 bits of quantum collision security under BHT, not 124 bits as a naive halving of the classical bound would suggest. A formal quantum security analysis of Poseidon2 over Mersenne-31 has not been established in the literature. Deployments requiring provable post-quantum collision resistance at a target security level κ should ensure capacity $\geq 3\kappa$ bits, and should treat the concrete instantiation as an open assumption pending dedicated cryptanalysis.

- **Privacy-throughput trade-off.** The choice between nonce and nullifier models involves a fundamental trade-off between ordering transparency and privacy. Hybrid models combining both strategies for different authorization classes merit further investigation.

14 Conclusion

ZK-ACE reframes post-quantum-ready blockchain authorization as an identity-bound proof problem rather than a signature-object verification problem. By replacing transaction-carried post-quantum signatures with zero-knowledge proofs of deterministic authorization, the resulting model preserves verifier-side semantic guarantees while eliminating consensus dependence on kilobyte-scale signature artifacts.

The construction achieves this through a minimal set of components: an identity commitment anchored on-chain, a circuit that enforces commitment consistency, target binding, transaction authorization, replay prevention, and domain separation, and a verification interface compatible with account-abstraction and rollup architectures. The formal security analysis demonstrates that authorization soundness, replay resistance, substitution resistance, and cross-domain separation all reduce to standard cryptographic assumptions.

The structural data accounting shows an order-of-magnitude reduction in per-transaction authorization data relative to direct post-quantum signature deployment—achieved not by compressing signatures, but by eliminating the need for signature objects in the consensus path entirely. A reference implementation with two pluggable backends confirms circuit sizes of 341 active rows / 361 AIR constraint expressions (Circle STARK) and 2,155 R1CS constraints (Groth16)—roughly $500\text{--}2,300\times$ smaller than in-circuit post-quantum signature verification. The STARK backend achieves 14.5 ms proving and 1.1 ms verification with transparent setup and post-quantum-oriented assumptions; under deployment models with mandatory per-block aggregation, per-transaction consensus-visible data is ≈ 160 bytes (public inputs) plus amortized aggregation proof overhead per block. The Groth16 backend provides 37.3 ms proving and 128-byte proofs for EVM-native deployments. The two replay semantics share the same circuit after squeeze2 unification, so benchmarks report the unified replay-circuit path. With support for batch aggregation and a path toward recursive composition, ZK-ACE provides a scalable authorization architecture for post-quantum blockchain systems.

Acknowledgments

The author thanks Alberto La Rocca for insightful discussion on sponge-based circuit minimization. Alberto La Rocca independently identified that Poseidon’s length-extension immunity makes HMAC-style double-hashing unnecessary in this setting, and confirmed a 2-call lower bound for the core MAC computation in his implementation. This observation directly motivated the 3-call optimization and the 2-call variant presented in Section 7.4.

References

- [1] G. Brassard, P. Høyer, and A. Tapp. Quantum cryptanalysis of hash and claw-free functions. In *LATIN 1998: Theoretical Informatics*, LNCS 1380, pp. 163–169. Springer, 1998. (Preprint: arXiv:quant-ph/9705002, 1997.)
- [2] J. S. Wang. ACE-GF: A Generative Framework for Atomic Cryptographic Entities. arXiv preprint arXiv:2511.20505, 2025. <https://arxiv.org/abs/2511.20505>.
- [3] NIST. FIPS 204: Module-Lattice-Based Digital Signature Standard (ML-DSA). Federal Information Processing Standards, 2024.
- [4] NIST. FIPS 205: Stateless Hash-Based Digital Signature Standard (SLH-DSA). Federal Information Processing Standards, 2024.
- [5] NIST. Draft FIPS 206: FFT over NTRU-Lattice-Based Digital Signature Standard (FN-DSA). Federal Information Processing Standards (draft), 2024.
- [6] A. Biryukov, D. Dinu, and D. Khovratovich. Argon2: The memory-hard function for password hashing and other applications. In *IEEE EuroS&P Workshops*, 2016.
- [7] L. Grassi, D. Kales, D. Khovratovich, A. Roy, C. Rechberger, and M. Schofnegger. Poseidon: A new hash function for zero-knowledge proof systems. In *USENIX Security Symposium*, 2021.
- [8] A. Gabizon, Z. J. Williamson, and O. Ciobotaru. PLONK: Permutations over Lagrange-bases for Oecumenical Noninteractive arguments of Knowledge. *IACR ePrint* 2019/953, 2019.
- [9] J. Groth. On the Size of Pairing-based Non-interactive Arguments. In *EUROCRYPT*, 2016.
- [10] E. Ben-Sasson, I. Bentov, Y. Horesh, and M. Riabzev. Scalable, transparent, and post-quantum secure computational integrity. *IACR ePrint* 2018/046, 2018.
- [11] B. Bünz, J. Bootle, D. Boneh, A. Poelstra, P. Wuille, and G. Maxwell. Bulletproofs: Short proofs for confidential transactions and more. In *IEEE S&P*, 2018.
- [12] S. Bowe, J. Grigg, and D. Hopwood. Recursive proof composition without a trusted setup. *IACR ePrint* 2019/1021, 2019.
- [13] A. Kothapalli, S. Setty, and I. Tzialla. Nova: Recursive zero-knowledge arguments from folding schemes. In *CRYPTO*, 2022.
- [14] E. B. Sasson, A. Chiesa, C. Garman, M. Green, I. Miers, E. Tromer, and M. Virza. Zero-cash: Decentralized anonymous payments from Bitcoin. In *IEEE S&P*, 2014.
- [15] A. Pertsev, R. Semenov, and R. Storm. Tornado Cash privacy solution. White Paper, 2019.
- [16] Privacy and Scaling Explorations (PSE), Ethereum Foundation. Semaphore: Zero-knowledge signaling on Ethereum. <https://semaphore.pse.dev/>, 2020–2024.
- [17] V. Buterin, Y. Weiss, D. Kristjansson, N. Luca, T. Silberstein, and M. Tirosh. ERC-4337: Account abstraction using alt mempool. Ethereum Improvement Proposal, 2021.
- [18] V. Buterin, D. Feist, D. Loerakker, G. Kadianakis, M. Garnett, M. Taiwo Adeniyi, and A. Schoedon. EIP-4844: Shard blob transactions. Ethereum Improvement Proposal, 2022.

- [19] 0xPARC. circom-ecdsa: Proving ECDSA with zkSNARKs. <https://github.com/0xPARC/circom-ecdsa>, 2022.
- [20] D. Boneh and M. Franklin. Identity-based encryption from the Weil pairing. In *CRYPTO*, 2001.
- [21] A. Shamir. Identity-based cryptosystems and signature schemes. In *CRYPTO*, 1984.
- [22] H. Krawczyk and P. Eronen. HMAC-based Extract-and-Expand Key Derivation Function (HKDF). RFC 5869, 2010.
- [23] S. Goldwasser, S. Micali, and C. Rackoff. The knowledge complexity of interactive proof systems. *SIAM Journal on Computing*, 18(1):186–208, 1989.
- [24] M. Bellare and P. Rogaway. The security of triple encryption and a framework for code-based game-playing proofs. In *EUROCRYPT*, 2006.
- [25] D. Boneh and V. Shoup. *A Graduate Course in Applied Cryptography*. Version 0.6, 2023. <https://toc.cryptobook.us/>.
- [26] A. Chiesa, G. Fenzi, and J. Weissenberg. Zero-Knowledge IOPPs for Constrained Interleaved Codes. *IACR ePrint* 2026/391, 2026.
- [27] StarkWare Industries. Stwo: A Circle STARK prover. <https://github.com/starkware-libs/stwo>, 2024–2026.
- [28] arkworks contributors. arkworks: An ecosystem for developing and programming with zkSNARKs. <https://arkworks.rs/>, 2021–2024.
- [29] Brook Heisler et al. Criterion.rs: Statistics-driven micro-benchmarking library for Rust. <https://bheisler.github.io/criterion.rs/book/>, 2017–2025.
- [30] G. Bertoni, J. Daemen, M. Peeters, and G. Van Assche. Cryptographic sponge functions. Submission to NIST, 2011. <https://sponge.noekeon.org/CSF-0.1.pdf>.
- [31] J. Czajkowski, L. Groot Bruinderink, A. Hülsing, C. Schaffner, and D. Unruh. Post-quantum security of the sponge construction. In *Post-Quantum Cryptography – PQCrypto 2019*, LNCS 11505, pp. 185–204. Springer, 2019.
- [32] D. Boneh, O. Dagdelen, M. Fischlin, A. Lehmann, C. Schaffner, and M. Zhandry. Random oracles in a quantum world. In *Advances in Cryptology – ASIACRYPT 2011*, LNCS 7073, pp. 41–69. Springer, 2011.
- [33] Alberto La Rocca. Reply to: What if post-quantum Ethereum doesn’t need signatures at all? <https://ethresear.ch/t/what-if-post-quantum-ethereum-doesn-t-need-signatures-at-all/24427/10>, May 2026. ethresear.ch forum comment.